

In [2]: `import pandas as pd`

```
df = pd.read_csv("housing.csv")
df.head()
```

Out[2]: `<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>longitude</th> <th>latitude</th> <th>housing_median_age</th> <th>total_rooms</th> <th>total_bedrooms</th> <th>population</th> <th>households</th> <th>median_income</th> <th>median_house_value</th> <th>ocean_proximity</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>`

-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY

In [3]: `df.head()`

Out[3]: `<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>longitude</th> <th>latitude</th> <th>housing_median_age</th> <th>total_rooms</th> <th>total_bedrooms</th> <th>population</th> <th>households</th> <th>median_income</th> <th>median_house_value</th> <th>ocean_proximity</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>`

-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY

In [4]: `df.head(10)`

```
Out[4]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>longitude</th> <th>latitude</th> <th>housing_median_age</th>
<th>total_rooms</th> <th>total_bedrooms</th> <th>population</th>
<th>households</th> <th>median_income</th> <th>median_house_value</th>
<th>ocean_proximity</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th>
<th>9</th> </tbody> <tbody></tbody>
```

-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY
-122.25	37.85	52.0	919.0	213.0	413.0	193.0	4.0368	269700.0	NEAR BAY
-122.25	37.84	52.0	2535.0	489.0	1094.0	514.0	3.6591	299200.0	NEAR BAY
-122.25	37.84	52.0	3104.0	687.0	1157.0	647.0	3.1200	241400.0	NEAR BAY
-122.26	37.84	42.0	2555.0	665.0	1206.0	595.0	2.0804	226700.0	NEAR BAY
-122.25	37.84	52.0	3549.0	707.0	1551.0	714.0	3.6912	261100.0	NEAR BAY

```
In [5]: df.tail()
```

```
Out[5]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>longitude</th> <th>latitude</th> <th>housing_median_age</th>
<th>total_rooms</th> <th>total_bedrooms</th> <th>population</th>
<th>households</th> <th>median_income</th> <th>median_house_value</th>
<th>ocean_proximity</th> </thead> <tbody> <th>20635</th> <th>20636</th>
<th>20637</th> <th>20638</th> <th>20639</th> </tbody> <tbody></tbody>
```

-121.09	39.48	25.0	1665.0	374.0	845.0	330.0	1.5603	78100.0	INLAND
-121.21	39.49	18.0	697.0	150.0	356.0	114.0	2.5568	77100.0	INLAND
-121.22	39.43	17.0	2254.0	485.0	1007.0	433.0	1.7000	92300.0	INLAND
-121.32	39.43	18.0	1860.0	409.0	741.0	349.0	1.8672	84700.0	INLAND
-121.24	39.37	16.0	2785.0	616.0	1387.0	530.0	2.3886	89400.0	INLAND

```
In [6]: df.shape
```

```
Out[6]: (20640, 10)
```

```
In [7]: df.columns
```

```
Out[7]: Index(['longitude', 'latitude', 'housing_median_age', 'total_rooms',
              'total_bedrooms', 'population', 'households', 'median_income',
              'median_house_value', 'ocean_proximity'],
              dtype='str')
```

```
In [8]: df.dtypes
```

```
Out[8]: longitude      float64
latitude      float64
housing_median_age    float64
total_rooms      float64
total_bedrooms      float64
population      float64
households      float64
median_income      float64
median_house_value  float64
ocean_proximity      str
dtype: object
```

```
In [9]: df.describe()
```

```
Out[9]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>longitude</th> <th>latitude</th> <th>housing_median_age</th>
<th>total_rooms</th> <th>total_bedrooms</th> <th>population</th>
<th>households</th> <th>median_income</th> <th>median_house_value</th>
</thead> <tbody> <th>count</th> <th>mean</th> <th>std</th> <th>min</th>
<th>25%</th> <th>50%</th> <th>75%</th> <th>max</th> </tbody> <tbody>
</tbody>
20640.000000  20640.000000  20640.000000  20640.000000  20433.000000  20640.000000  20
-119.569704   35.631861   28.639486   2635.763081   537.870553   1425.476744
    2.003532    2.135952   12.585558   2181.615252   421.385070   1132.462122
-124.350000   32.540000    1.000000    2.000000    1.000000    3.000000
-121.800000   33.930000   18.000000   1447.750000   296.000000   787.000000
-118.490000   34.260000   29.000000   2127.000000   435.000000   1166.000000
-118.010000   37.710000   37.000000   3148.000000   647.000000   1725.000000
-114.310000   41.950000   52.000000  39320.000000  6445.000000  35682.000000  6
```

```
In [10]: df.isnull().sum()
```

```
Out[10]: longitude      0
latitude      0
housing_median_age  0
total_rooms    0
total_bedrooms 207
population     0
households     0
median_income  0
median_house_value  0
ocean_proximity  0
dtype: int64
```

```
In [11]: df.duplicated().sum()
```

```
Out[11]: np.int64(0)
```

```
In [12]: df.corr(numeric_only=True)
```

```
Out[12]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>longitude</th> <th>latitude</th> <th>housing_median_age</th>
<th>total_rooms</th> <th>total_bedrooms</th> <th>population</th>
<th>households</th> <th>median_income</th> <th>median_house_value</th>
</thead> <tbody> <th>longitude</th> <th>latitude</th>
<th>housing_median_age</th> <th>total_rooms</th> <th>total_bedrooms</th>
<th>population</th> <th>households</th> <th>median_income</th>
<th>median_house_value</th> </tbody> <tbody></tbody>

1.000000 -0.924664 -0.108197  0.044568  0.069608  0.099773  0.055310 -0.015176 -0.045967
-0.924664  1.000000  0.011173 -0.036100 -0.066983 -0.108785 -0.071035 -0.079809 -0.144160
-0.108197  0.011173  1.000000 -0.361262 -0.320451 -0.296244 -0.302916 -0.119034  0.105623
 0.044568 -0.036100 -0.361262  1.000000  0.930380  0.857126  0.918484  0.198050  0.134153
 0.069608 -0.066983 -0.320451  0.930380  1.000000  0.877747  0.979728 -0.007723  0.049686
 0.099773 -0.108785 -0.296244  0.857126  0.877747  1.000000  0.907222  0.004834 -0.024650
 0.055310 -0.071035 -0.302916  0.918484  0.979728  0.907222  1.000000  0.013033  0.065843
-0.015176 -0.079809 -0.119034  0.198050 -0.007723  0.004834  0.013033  1.000000  0.688075
-0.045967 -0.144160  0.105623  0.134153  0.049686 -0.024650  0.065843  0.688075  1.000000

<img alt="Horizontal scrollbar" data-bbox="195 755 820 768"/>
```

```
In [13]: plt.figure(figsize=(12,8))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.show()
```

```

-----
NameError                                Traceback (most recent call last)
Cell In[13], line 1
----> 1 plt.figure(figsize=(12,8))
      2 sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
      3 plt.show()

NameError: name 'plt' is not defined

```

```

In [14]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

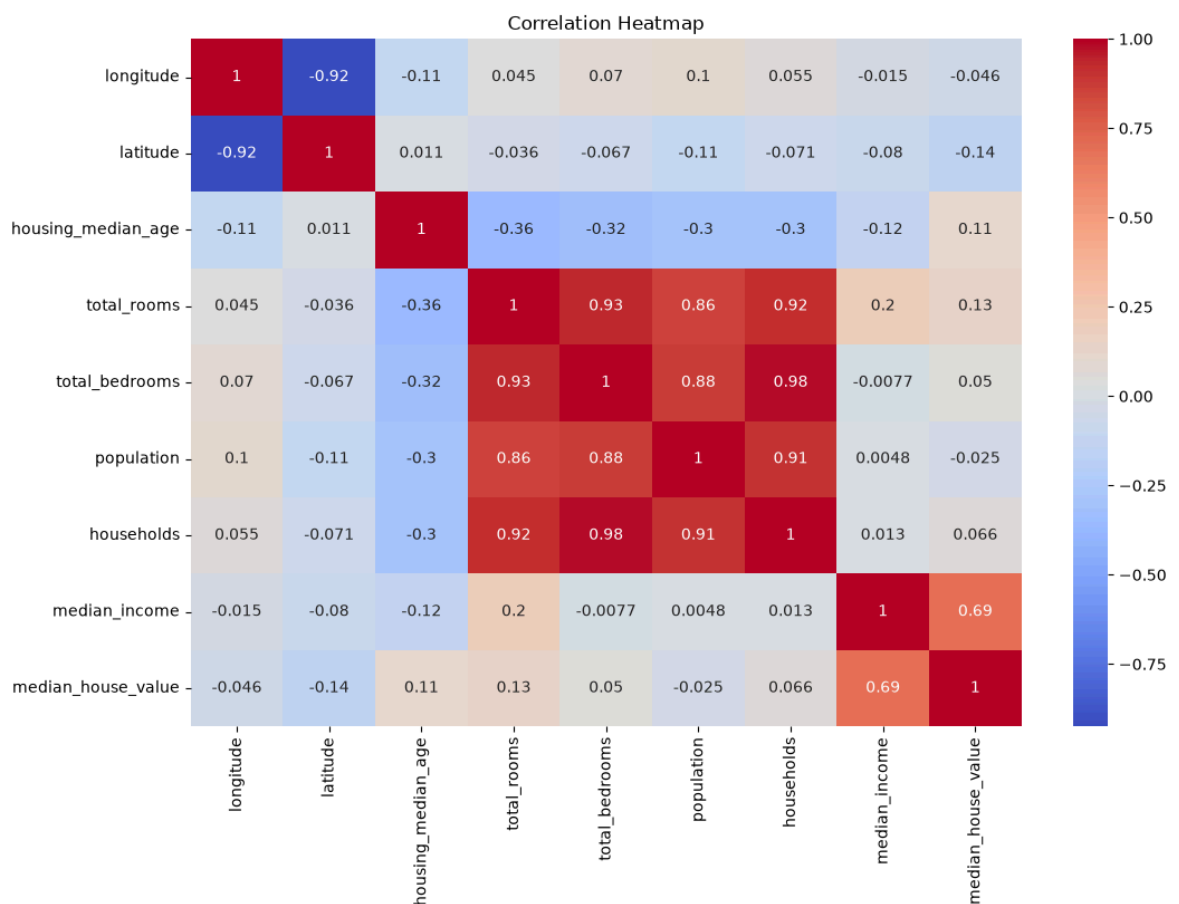
%matplotlib inline

```

```

In [15]: plt.figure(figsize=(12, 8))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()

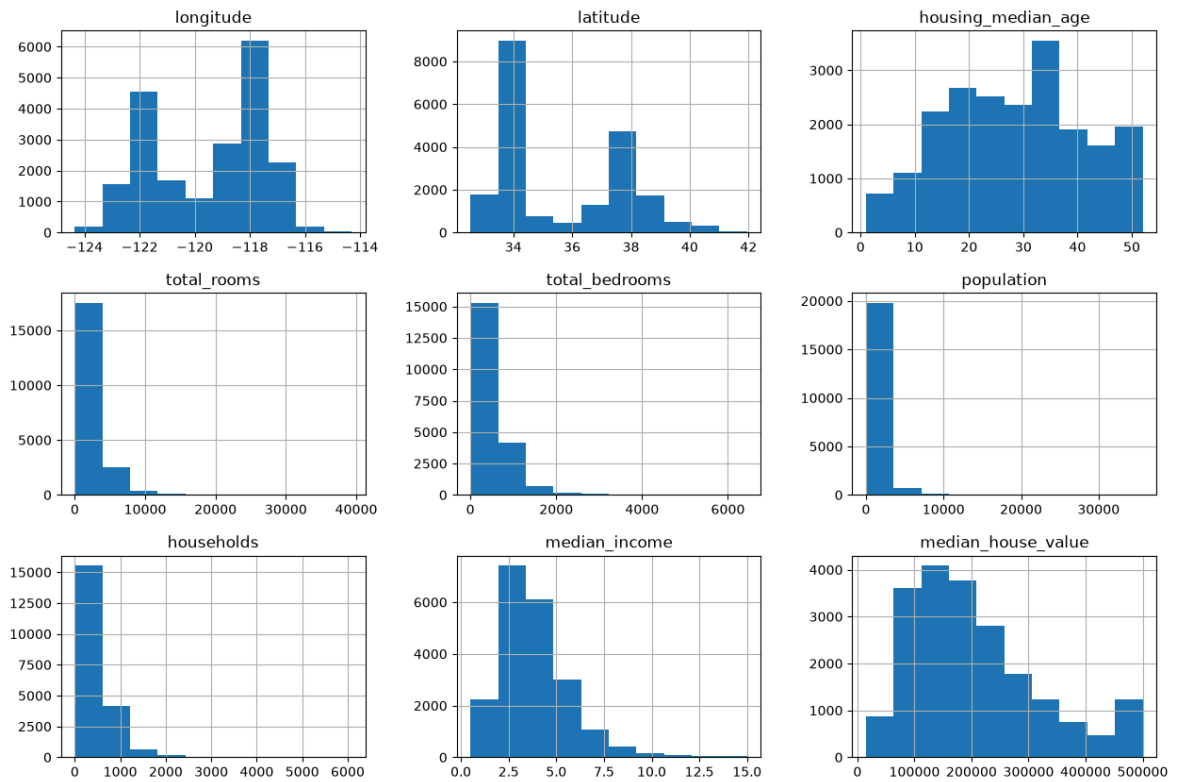
```



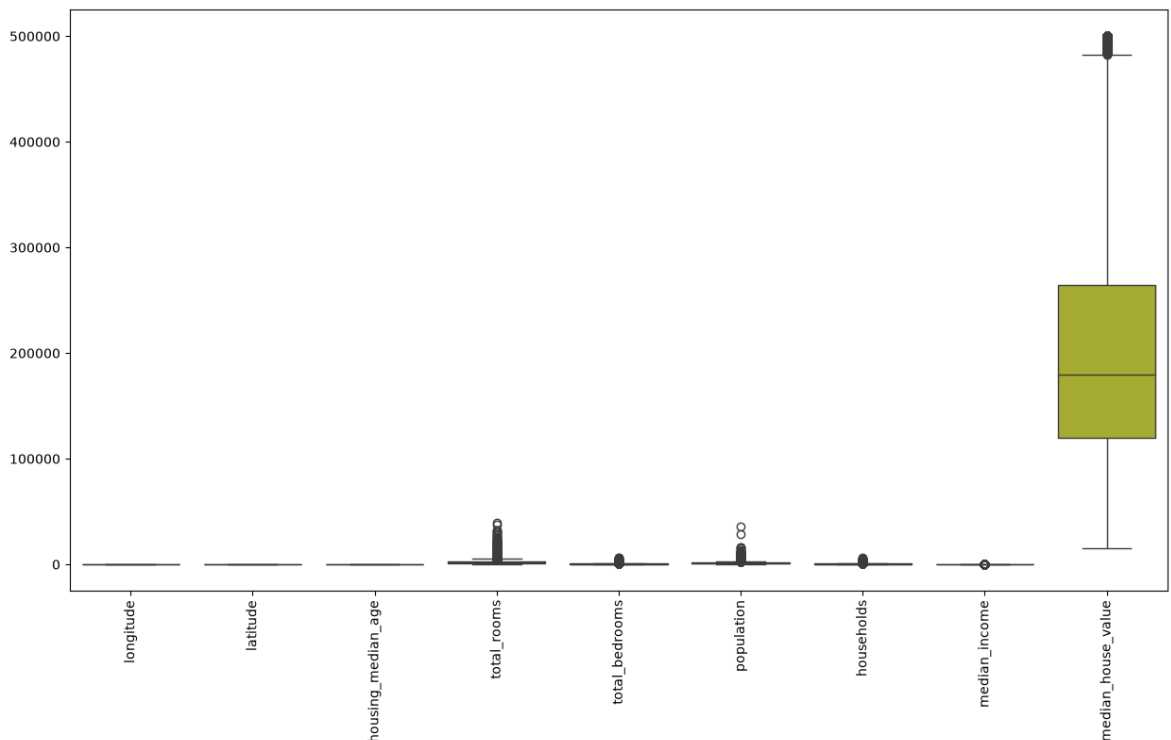
```

In [16]: df.hist(figsize=(15,10))
plt.show()

```



```
In [17]: plt.figure(figsize=(15,8))
sns.boxplot(data=df)
plt.xticks(rotation=90)
plt.show()
```



```
In [18]: numerical_columns = df.select_dtypes(include=['int64', 'float64']).columns
categorical_columns = df.select_dtypes(include=['object']).columns

print("Numerical Columns:")
print(numerical_columns)
```

```
print("\nCategorical Columns:")
print(categorical_columns)
```

Numerical Columns:

```
Index(['longitude', 'latitude', 'housing_median_age', 'total_rooms',
      'total_bedrooms', 'population', 'households', 'median_income',
      'median_house_value'],
      dtype='str')
```

Categorical Columns:

```
Index(['ocean_proximity'], dtype='str')
```

C:\Users\sandhya.kulkarni\AppData\Local\Temp\ipykernel_15604\2041061977.py:2: PandasWarning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.

See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.

```
categorical_columns = df.select_dtypes(include=['object']).columns
```

```
In [19]: numerical_columns = df.select_dtypes(include=['number']).columns
categorical_columns = df.select_dtypes(include=['object', 'string']).columns

print("Numerical Columns:")
print(numerical_columns)

print("\nCategorical Columns:")
print(categorical_columns)
```

Numerical Columns:

```
Index(['longitude', 'latitude', 'housing_median_age', 'total_rooms',
      'total_bedrooms', 'population', 'households', 'median_income',
      'median_house_value'],
      dtype='str')
```

Categorical Columns:

```
Index(['ocean_proximity'], dtype='str')
```

```
In [20]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   longitude             20640 non-null  float64
 1   latitude              20640 non-null  float64
 2   housing_median_age    20640 non-null  float64
 3   total_rooms           20640 non-null  float64
 4   total_bedrooms        20433 non-null  float64
 5   population            20640 non-null  float64
 6   households            20640 non-null  float64
 7   median_income         20640 non-null  float64
 8   median_house_value    20640 non-null  float64
 9   ocean_proximity       20640 non-null  str
dtypes: float64(9), str(1)
memory usage: 1.6 MB
```

```
In [21]: df.isnull().sum()
```

```
Out[21]: longitude      0
latitude      0
housing_median_age  0
total_rooms   0
total_bedrooms 207
population    0
households    0
median_income  0
median_house_value  0
ocean_proximity  0
dtype: int64
```

```
In [22]: df.isnull().sum()
```

```
Out[22]: longitude      0
latitude      0
housing_median_age  0
total_rooms   0
total_bedrooms 207
population    0
households    0
median_income  0
median_house_value  0
ocean_proximity  0
dtype: int64
```

```
In [23]: df["total_bedrooms"] = df["total_bedrooms"].fillna(df["total_bedrooms"].median())
```

```
In [24]: df.isnull().sum()
```

```
Out[24]: longitude      0
latitude      0
housing_median_age  0
total_rooms   0
total_bedrooms  0
population    0
households    0
median_income  0
median_house_value  0
ocean_proximity  0
dtype: int64
```

```
In [25]: df.duplicated().sum()
```

```
Out[25]: np.int64(0)
```

```
In [26]: df = df.drop_duplicates()
```

```
In [27]: df.duplicated().sum()
```

```
Out[27]: np.int64(0)
```

```
In [28]: (df.select_dtypes(include=['number']) < 0).sum()
```



```
Out[28]: longitude          20640
latitude                   0
housing_median_age         0
total_rooms                0
total_bedrooms             0
population                 0
households                 0
median_income              0
median_house_value         0
dtype: int64
```

```
In [29]: (df["median_house_value"] == 0).sum()
```

```
Out[29]: np.int64(0)
```

```
In [30]: (df["housing_median_age"] == 0).sum()
```

```
Out[30]: np.int64(0)
```

```
In [31]: df.dtypes
```

```
Out[31]: longitude          float64
latitude                   float64
housing_median_age         float64
total_rooms                float64
total_bedrooms             float64
population                 float64
households                 float64
median_income              float64
median_house_value         float64
ocean_proximity            str
dtype: object
```

```
In [32]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   longitude              20640 non-null  float64
1   latitude               20640 non-null  float64
2   housing_median_age     20640 non-null  float64
3   total_rooms            20640 non-null  float64
4   total_bedrooms         20640 non-null  float64
5   population              20640 non-null  float64
6   households              20640 non-null  float64
7   median_income          20640 non-null  float64
8   median_house_value     20640 non-null  float64
9   ocean_proximity        20640 non-null  str
dtypes: float64(9), str(1)
memory usage: 1.6 MB
```

```
In [33]: df.shape
```

```
Out[33]: (20640, 10)
```

```
In [34]: df.mean(numeric_only=True)
```

```
Out[34]: longitude          -119.569704
latitude             35.631861
housing_median_age    28.639486
total_rooms           2635.763081
total_bedrooms        536.838857
population            1425.476744
households            499.539680
median_income          3.870671
median_house_value    206855.816909
dtype: float64
```

```
In [35]: df.median(numeric_only=True)
```

```
Out[35]: longitude          -118.4900
latitude             34.2600
housing_median_age    29.0000
total_rooms           2127.0000
total_bedrooms        435.0000
population            1166.0000
households            409.0000
median_income          3.5348
median_house_value    179700.0000
dtype: float64
```

```
In [36]: df.mode().iloc[0]
```

```
Out[36]: longitude          -118.31
latitude             34.06
housing_median_age    52.0
total_rooms           1527.0
total_bedrooms        435.0
population            891.0
households            306.0
median_income          3.125
median_house_value    500001.0
ocean_proximity       <1H OCEAN
Name: 0, dtype: object
```

```
In [37]: df.var(numeric_only=True)
```

```
Out[37]: longitude          4.014139e+00
latitude             4.562293e+00
housing_median_age    1.583963e+02
total_rooms           4.759445e+06
total_bedrooms        1.758895e+05
population            1.282470e+06
households            1.461760e+05
median_income          3.609323e+00
median_house_value    1.331615e+10
dtype: float64
```

```
In [38]: df.std(numeric_only=True)
```

```
Out[38]: longitude      2.003532
latitude      2.135952
housing_median_age  12.585558
total_rooms    2181.615252
total_bedrooms  419.391878
population     1132.462122
households     382.329753
median_income   1.899822
median_house_value 115395.615874
dtype: float64
```

```
In [39]: Q1 = df.quantile(0.25, numeric_only=True)
Q3 = df.quantile(0.75, numeric_only=True)
IQR = Q3 - Q1

print(IQR)
```

```
longitude      3.79000
latitude      3.78000
housing_median_age  19.00000
total_rooms    1700.25000
total_bedrooms  346.25000
population     938.00000
households     325.00000
median_income   2.17985
median_house_value 145125.00000
dtype: float64
```

```
In [40]: summary = pd.DataFrame({
    "Mean": df.mean(numeric_only=True),
    "Median": df.median(numeric_only=True)
})

summary["Skewness"] = np.where(
    summary["Mean"] > summary["Median"],
    "Right Skewed",
    np.where(summary["Mean"] < summary["Median"],
             "Left Skewed",
             "Symmetric")
)

summary
```

```
Out[40]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>Mean</th> <th>Median</th> <th>Skewness</th> </thead> <tbody>
<th>longitude</th> <th>latitude</th> <th>housing_median_age</th>
<th>total_rooms</th> <th>total_bedrooms</th> <th>population</th>
<th>households</th> <th>median_income</th> <th>median_house_value</th>
</tbody> <tbody></tbody>
```

-119.569704	-118.4900	Left Skewed
35.631861	34.2600	Right Skewed
28.639486	29.0000	Left Skewed
2635.763081	2127.0000	Right Skewed
536.838857	435.0000	Right Skewed
1425.476744	1166.0000	Right Skewed
499.539680	409.0000	Right Skewed
3.870671	3.5348	Right Skewed
206855.816909	179700.0000	Right Skewed

```
In [41]: summary_table = pd.DataFrame({
    "Mean": df.mean(numeric_only=True),
    "Median": df.median(numeric_only=True),
    "Variance": df.var(numeric_only=True),
    "Std Dev": df.std(numeric_only=True),
    "Minimum": df.min(numeric_only=True),
    "Maximum": df.max(numeric_only=True)
})
```

summary_table

```
Out[41]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>Mean</th> <th>Median</th> <th>Variance</th> <th>Std Dev</th>
<th>Minimum</th> <th>Maximum</th> </thead> <tbody> <th>longitude</th>
<th>latitude</th> <th>housing_median_age</th> <th>total_rooms</th>
<th>total_bedrooms</th> <th>population</th> <th>households</th>
<th>median_income</th> <th>median_house_value</th> </tbody> <tbody></tbody>
```

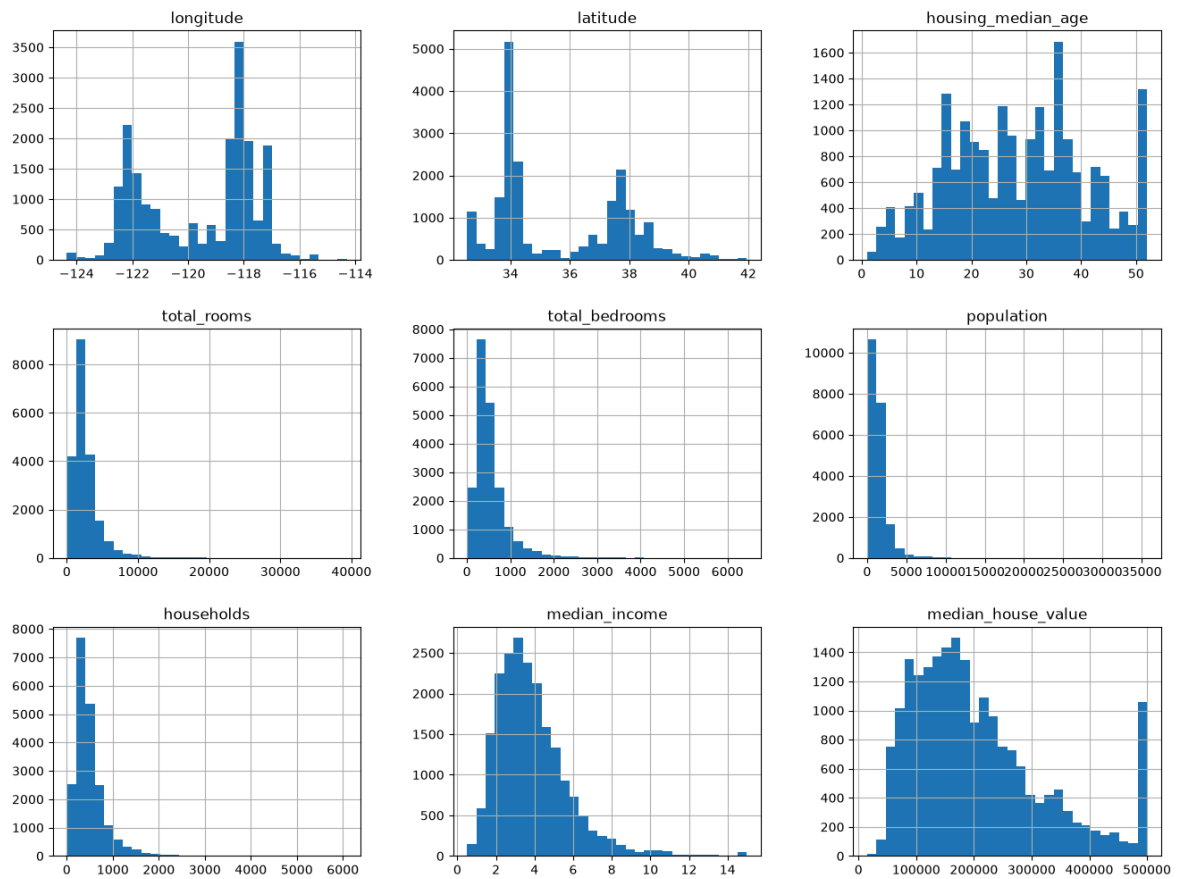
-119.569704	-118.4900	4.014139e+00	2.003532	-124.3500	-114.3100
35.631861	34.2600	4.562293e+00	2.135952	32.5400	41.9500
28.639486	29.0000	1.583963e+02	12.585558	1.0000	52.0000
2635.763081	2127.0000	4.759445e+06	2181.615252	2.0000	39320.0000
536.838857	435.0000	1.758895e+05	419.391878	1.0000	6445.0000
1425.476744	1166.0000	1.282470e+06	1132.462122	3.0000	35682.0000
499.539680	409.0000	1.461760e+05	382.329753	1.0000	6082.0000
3.870671	3.5348	3.609323e+00	1.899822	0.4999	15.0001
206855.816909	179700.0000	1.331615e+10	115395.615874	14999.0000	500001.0000

```
In [42]: df.hist(figsize=(16,12), bins=30)

plt.suptitle("Histogram of Numerical Features", fontsize=16)

plt.show()
```

Histogram of Numerical Features

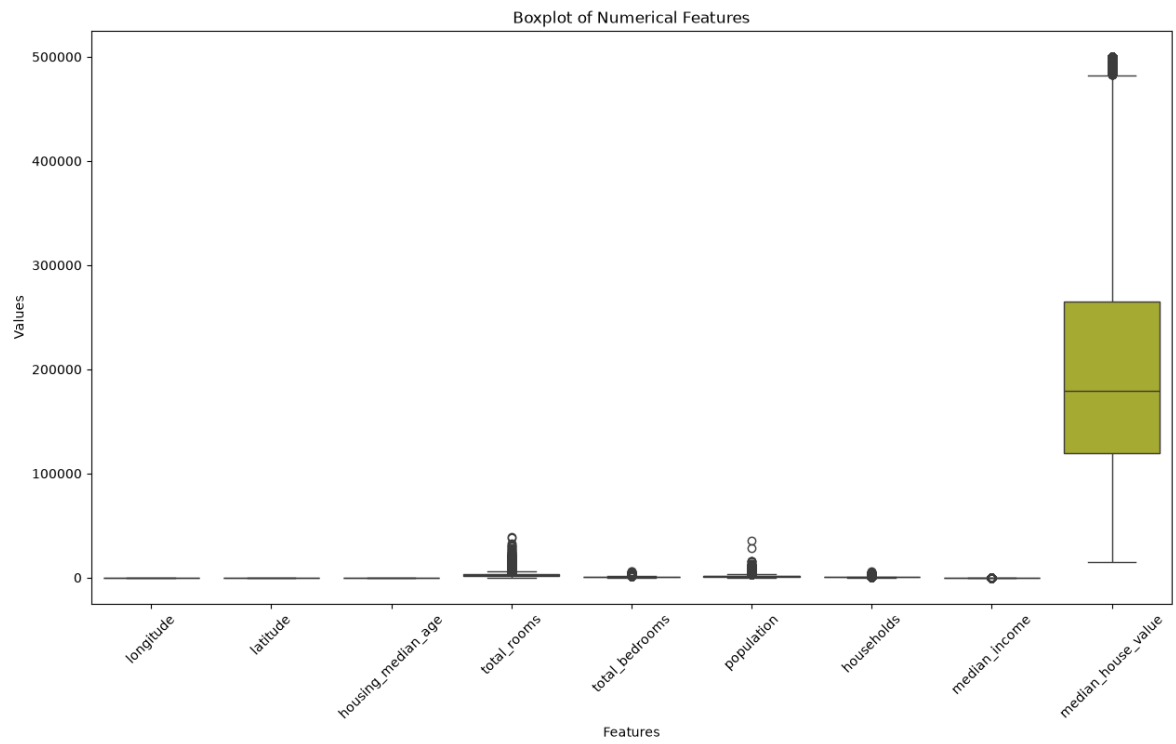


```
In [43]: plt.figure(figsize=(15,8))

sns.boxplot(data=df.select_dtypes(include="number"))

plt.title("Boxplot of Numerical Features")
plt.xticks(rotation=45)
plt.xlabel("Features")
plt.ylabel("Values")

plt.show()
```

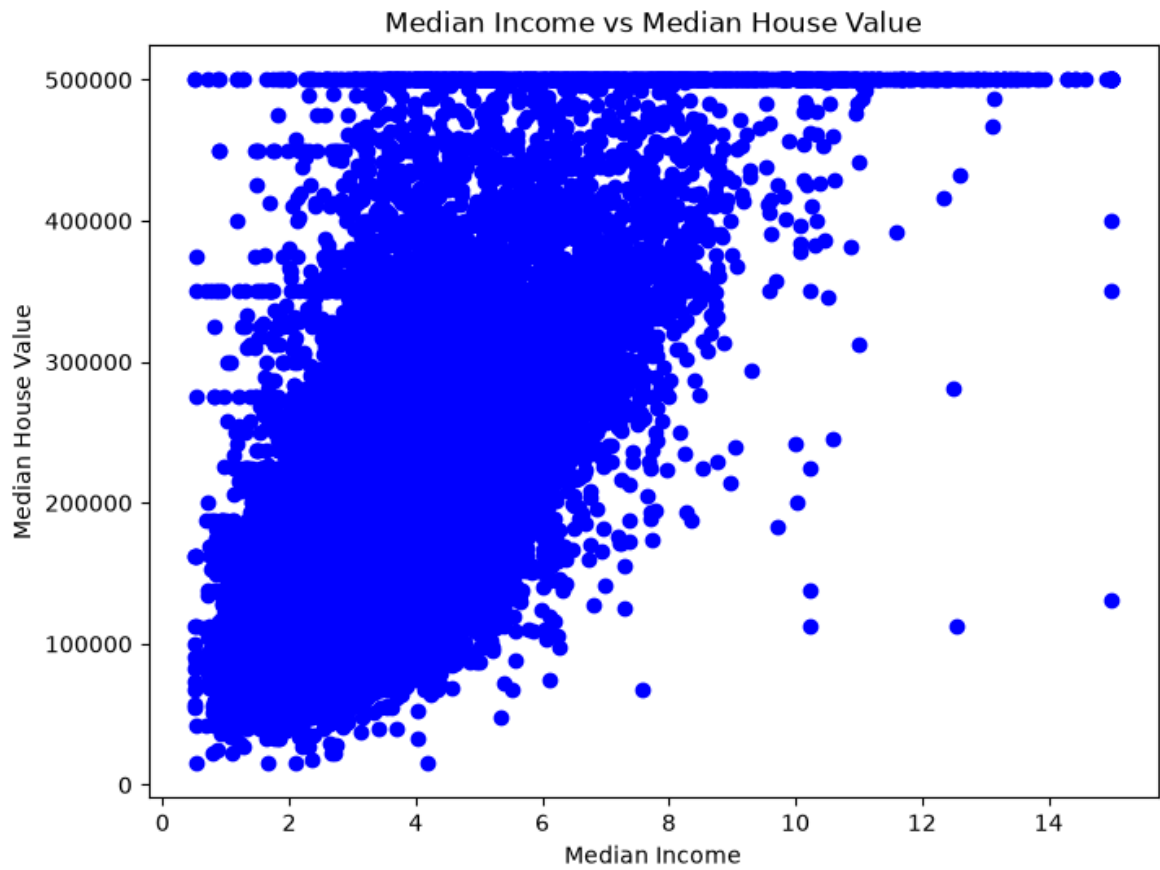


```
In [44]: plt.figure(figsize=(8,6))

plt.scatter(df["median_income"],
            df["median_house_value"],
            color="blue")

plt.title("Median Income vs Median House Value")
plt.xlabel("Median Income")
plt.ylabel("Median House Value")

plt.show()
```

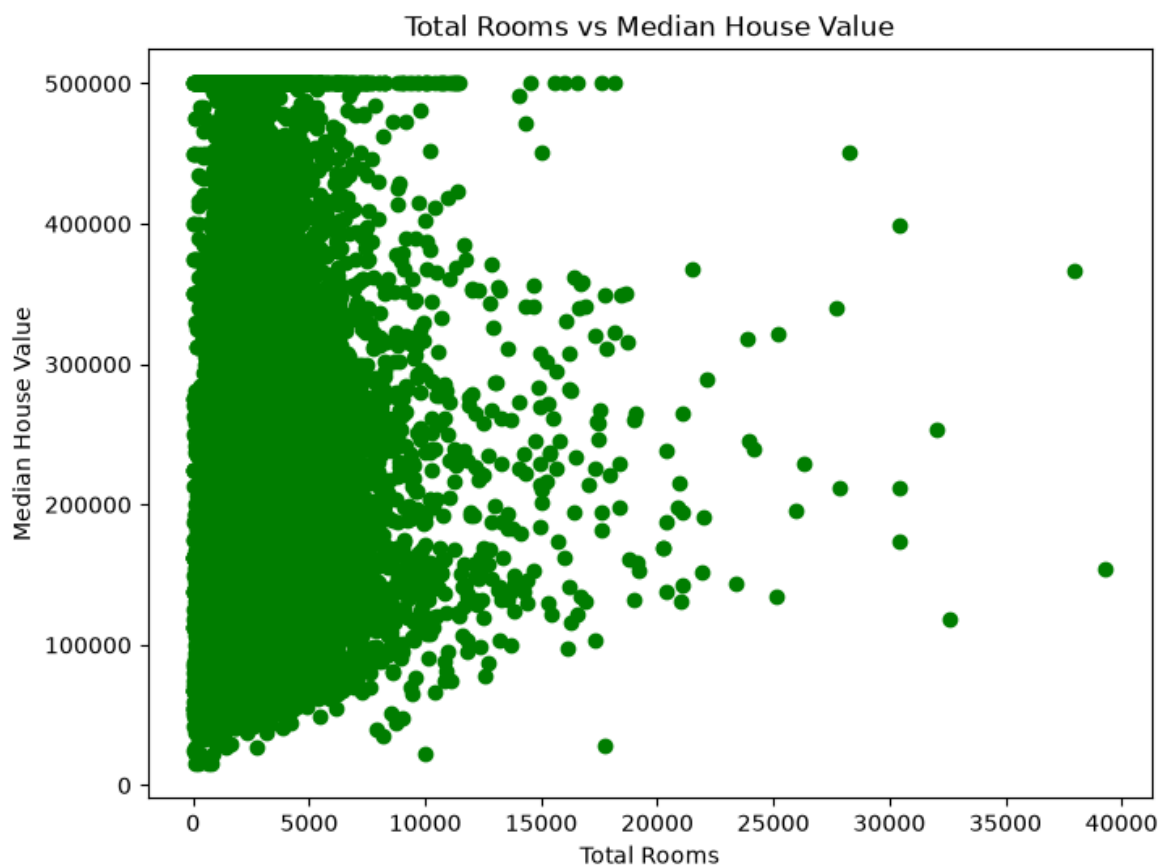


```
In [45]: plt.figure(figsize=(8,6))

plt.scatter(df["total_rooms"],
            df["median_house_value"],
            color="green")

plt.title("Total Rooms vs Median House Value")
plt.xlabel("Total Rooms")
plt.ylabel("Median House Value")

plt.show()
```

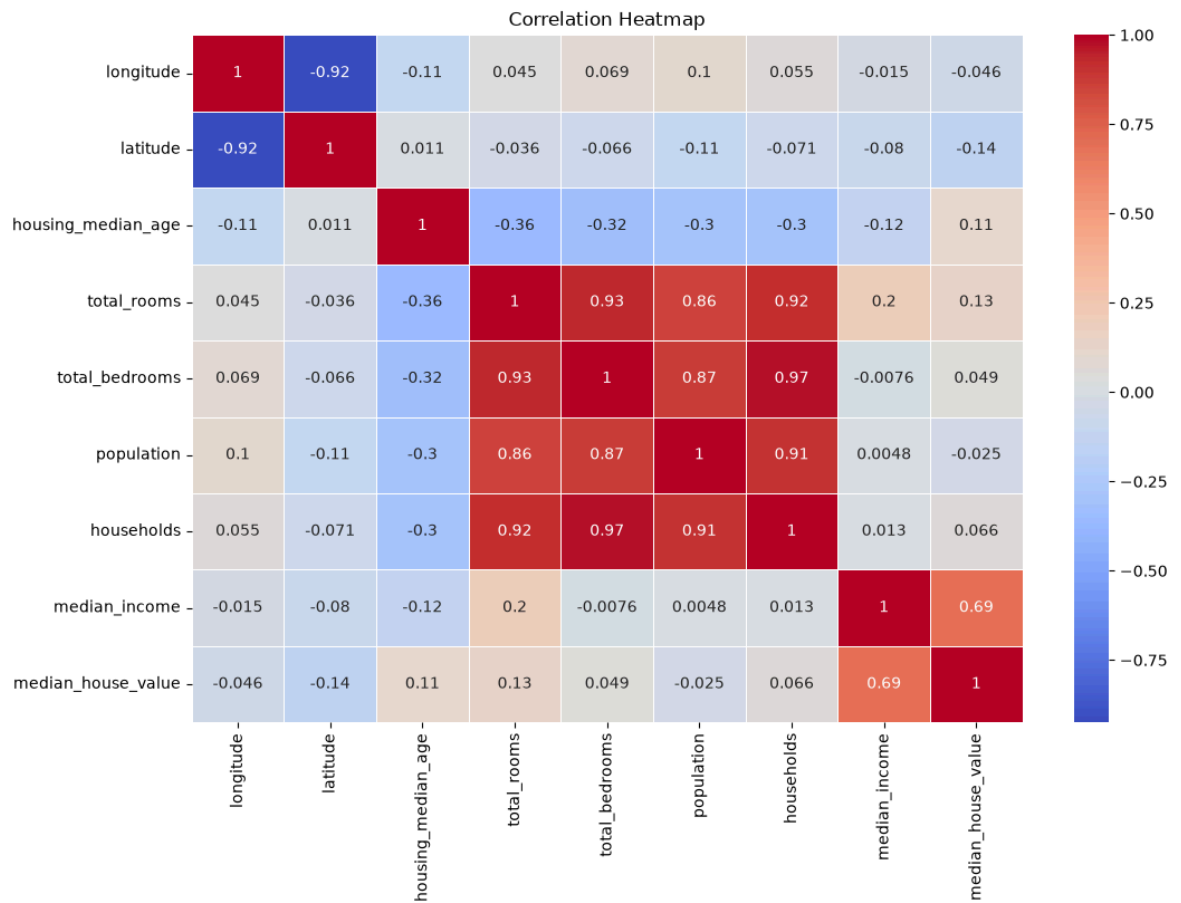



```
In [46]: plt.figure(figsize=(12,8))

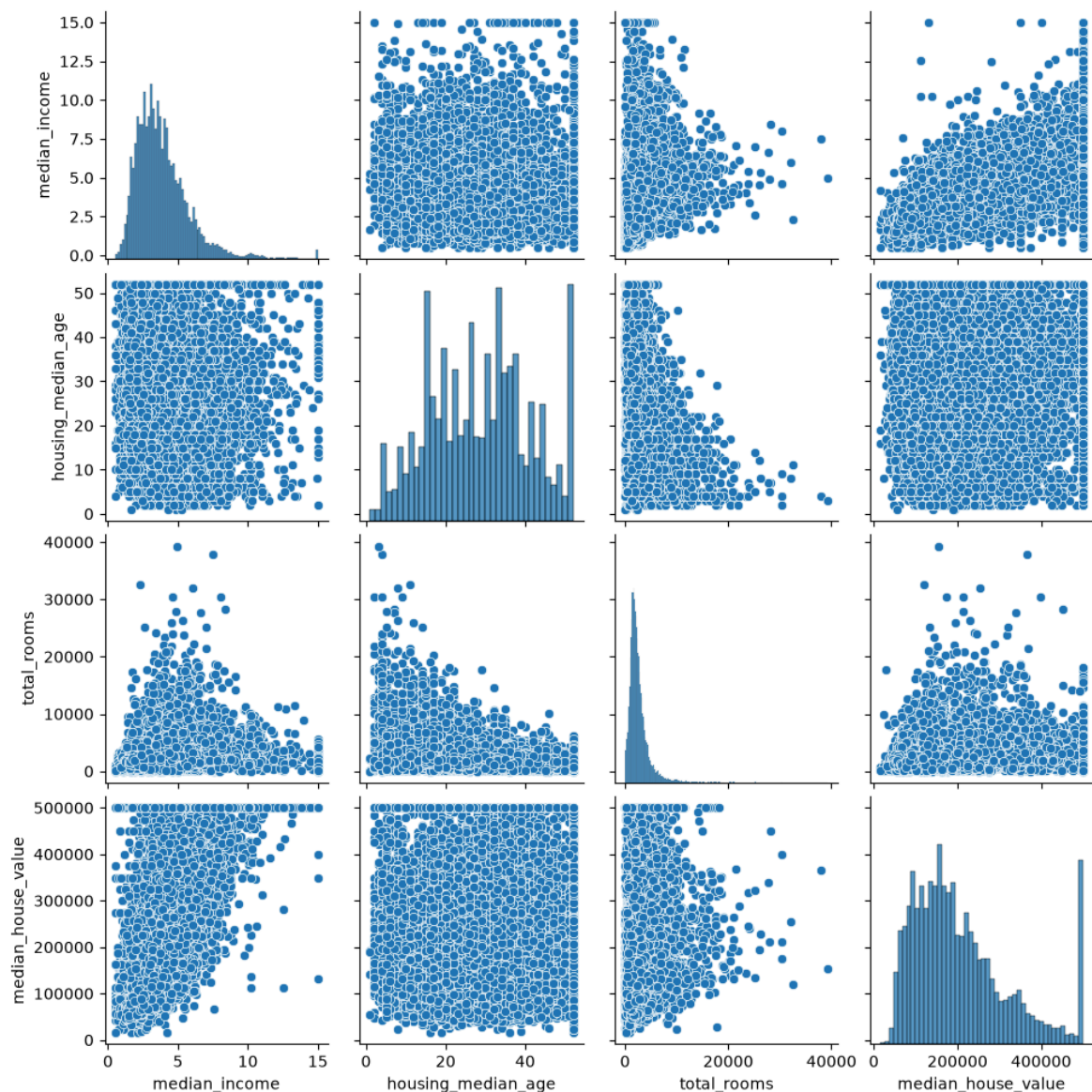
sns.heatmap(df.corr(numeric_only=True),
            annot=True,
            cmap="coolwarm",
            linewidths=0.5)

plt.title("Correlation Heatmap")

plt.show()
```



```
In [47]: sns.pairplot(
df[[
    "median_income",
    "housing_median_age",
    "total_rooms",
    "median_house_value"
]])
plt.show()
```



```
In [48]: X = df.drop(["median_house_value", "ocean_proximity"], axis=1)
y = df["median_house_value"]
```

```
In [49]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
-----
ModuleNotFoundError                                Traceback (most recent call last)
Cell In[49], line 1
----> 1 from sklearn.model_selection import train_test_split
      2
      3 X_train, X_test, y_train, y_test = train_test_split(
      4     X,

ModuleNotFoundError: No module named 'sklearn'
```

```
In [50]: from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```
In [51]: X = df.drop(["median_house_value", "ocean_proximity"], axis=1)
y = df["median_house_value"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
In [52]: from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = mse ** 0.5
r2 = r2_score(y_test, y_pred)

print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

```
MAE : 51810.483628042726
MSE : 5059928371.165331
RMSE: 71133.17349286008
R2 Score: 0.6138664756435196
```

```
In [53]: results = pd.DataFrame({
    "Actual": y_test,
    "Predicted": y_pred
})

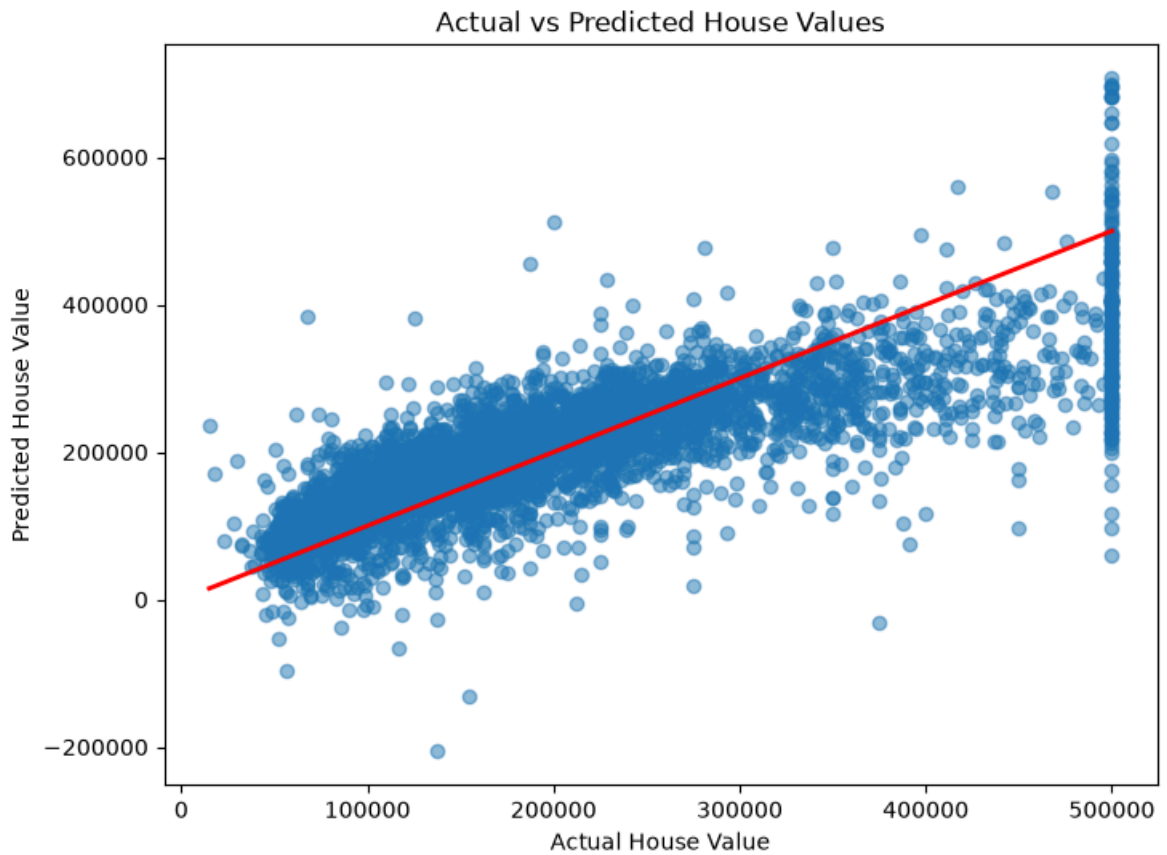
results.head(10)
```

```
Out[53]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>Actual</th> <th>Predicted</th> </thead> <tbody> <th>20046</th>
<th>3024</th> <th>15663</th> <th>20484</th> <th>9814</th> <th>13311</th>
<th>7113</th> <th>7668</th> <th>18246</th> <th>5723</th> </tbody> <tbody>
</tbody>
```

47700.0	63736.591338
45800.0	154344.594319
500001.0	253073.194287
218600.0	263507.746536
278000.0	266883.359611
158700.0	155403.494466
198200.0	287058.654200
157500.0	223660.781552
340000.0	257045.027183
446600.0	404273.366113

```
In [54]: import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    color='red',
    linewidth=2
)
plt.title("Actual vs Predicted House Values")
plt.xlabel("Actual House Value")
plt.ylabel("Predicted House Value")
plt.show()
```



```
In [55]: median_income
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[55], line 1  
----> 1 median_income  
  
NameError: name 'median_income' is not defined
```

```
In [56]: X = df[['median_income']]  
        y = df['median_house_value']
```

```
In [57]: from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

```
In [58]: from sklearn.linear_model import LinearRegression  
  
simple_model = LinearRegression()  
simple_model.fit(X_train, y_train)
```

```

Out[58]: <style>.sk-global { /* Definition of color scheme common for light and dark mode */ --
sklearn-color-text: #000; --sklearn-color-text-muted: #666; --sklearn-color-line: gray; /*
Definition of color scheme for unfitted estimators */ --sklearn-color-unfitted-level-0:
#fff5e6; --sklearn-color-unfitted-level-1: #f6e4d2; --sklearn-color-unfitted-level-2:
#ffe0b3; --sklearn-color-unfitted-level-3: chocolate; /* Definition of color scheme for
fitted estimators */ --sklearn-color-fitted-level-0: #f0f8ff; --sklearn-color-fitted-level-1:
#d4ebff; --sklearn-color-fitted-level-2: #b3dbfd; --sklearn-color-fitted-level-3:
cornflowerblue; } .sk-global.light { /* Specific color for light theme */ --sklearn-color-text-
on-default-background: black; --sklearn-color-background: white; --sklearn-color-border-
box: black; --sklearn-color-icon: #696969; } .sk-global.dark { --sklearn-color-text-on-
default-background: white; --sklearn-color-background: #111; --sklearn-color-border-
box: white; --sklearn-color-icon: #878787; } .sk-global { color: var(--sklearn-color-text); }
.sk-global pre { padding: 0; } .sk-global input.sk-hidden--visually { border: 0; clip-path:
inset(100%); height: 1px; margin: -1px; overflow: hidden; padding: 0; position: absolute;
width: 1px; } .sk-global div.sk-dashed-wrapped { border: 1px dashed var(--sklearn-color-
line); margin: 0 0.4em 0.5em 0.4em; box-sizing: border-box; padding-bottom: 0.4em;
background-color: var(--sklearn-color-background); } .sk-global div.sk-container { /*
jupyter's `normalize.less` sets `[hidden] { display: none; }` but bootstrap.min.css set
`[hidden] { display: none !important; }` so we also need the `!important` here to be able to
override the default hidden behavior on the sphinx rendered scikit-learn.org. See:
https://github.com/scikit-learn/scikit-learn/issues/21755 */ display: inline-block
!important; position: relative; } .sk-global div.sk-text-repr-fallback { display: none; } div.sk-
parallel-item, div.sk-serial, div.sk-item { /* draw centered vertical line to link estimators */
background-image: linear-gradient(var(--sklearn-color-text-on-default-background), var(-
sklearn-color-text-on-default-background)); background-size: 2px 100%; background-
repeat: no-repeat; background-position: center center; } /* Parallel-specific style estimator
block */ .sk-global div.sk-parallel-item::after { content: ""; width: 100%; border-bottom:
2px solid var(--sklearn-color-text-on-default-background); flex-grow: 1; } .sk-global div.sk-
parallel { display: flex; align-items: stretch; justify-content: center; background-color: var(--
sklearn-color-background); position: relative; } .sk-global div.sk-parallel-item { display: flex;
flex-direction: column; } .sk-global div.sk-parallel-item:first-child::after { align-self: flex-
end; width: 50%; } .sk-global div.sk-parallel-item:last-child::after { align-self: flex-start;
width: 50%; } .sk-global div.sk-parallel-item:only-child::after { width: 0; } /* Serial-specific
style estimator block */ .sk-global div.sk-serial { display: flex; flex-direction: column; align-
items: center; background-color: var(--sklearn-color-background); padding-right: 1em;
padding-left: 1em; } /* Toggleable style: style used for
estimator/Pipeline/ColumnTransformer box that is clickable and can be
expanded/collapsed. - Pipeline and ColumnTransformer use this feature and define the
default style - Estimators will overwrite some part of the style using the `sk-estimator`
class */ /* Pipeline and ColumnTransformer style (default) */ .sk-global div.sk-toggleable {
/* Default theme specific background. It is overwritten whether we have a specific
estimator or a Pipeline/ColumnTransformer */ background-color: var(--sklearn-color-
background); } /* Toggleable label */ .sk-global label.sk-toggleable__label { cursor: pointer;
display: flex; width: 100%; margin-bottom: 0; padding: 0.5em; box-sizing: border-box;
text-align: center; align-items: center; justify-content: center; gap: 0.5em; } .sk-global

```

```

label.sk-toggleable__label.caption { font-size: 0.6rem; font-weight: lighter; color: var(--sklearn-color-text-muted); } .sk-global label.sk-toggleable__label-arrow:before { /* Arrow on the left of the label */ content: " ▶ "; float: left; margin-right: 0.25em; color: var(--sklearn-color-icon); } .sk-global label.sk-toggleable__label-arrow:hover:before { color: var(--sklearn-color-text); } /* Toggleable content - dropdown */ .sk-global div.sk-toggleable__content { display: none; text-align: left; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global div.sk-toggleable__content pre { margin: 0.2em; border-radius: 0.25em; color: var(--sklearn-color-text); /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted pre { /* unfitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global input.sk-toggleable__control:checked~div.sk-toggleable__content { /* Expand drop-down */ display: block; width: 100%; overflow: visible; } .sk-global input.sk-toggleable__control:checked~label.sk-toggleable__label-arrow:before { content: " ▼ "; } /* Pipeline/ColumnTransformer-specific style */ .sk-global div.sk-label input.sk-toggleable__control:checked~label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-label.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { background-color: var(--sklearn-color-fitted-level-2); } /* Estimator-specific style */ /* Colorize estimator box */ .sk-global div.sk-estimator input.sk-toggleable__control:checked~label.sk-toggleable__label { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } .sk-global div.sk-label label.sk-toggleable__label, .sk-global div.sk-label label { /* The background is the default theme color */ color: var(--sklearn-color-text-on-default-background); } /* On hover, darken the color of the background */ .sk-global div.sk-label:hover label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } /* Label box, darken color on hover, fitted */ .sk-global div.sk-label.fitted:hover label.sk-toggleable__label.fitted { color: var(--sklearn-color-text); background-color: var(--sklearn-color-fitted-level-2); } /* Estimator label */ .sk-global div.sk-label label { font-family: monospace; font-weight: bold; line-height: 1.2em; } .sk-global div.sk-label-container { text-align: center; } /* Estimator-specific */ .sk-global div.sk-estimator { font-family: monospace; border: 1px dotted var(--sklearn-color-border-box); border-radius: 0.25em; box-sizing: border-box; margin-bottom: 0.5em; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-estimator.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } /* on hover */ .sk-global div.sk-estimator:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } /* Specification for estimator info (e.g. "i" and "?") */ /* Common style for "i" and "?" */ .sk-estimator-doc-link, a.link.sk-estimator-doc-link, a.visited.sk-estimator-doc-link { float: right; font-size: smaller; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1em; height: 1em; width: 1em; text-decoration: none !important; margin-left: 0.5em; text-align: center; /* unfitted */ border: var(--sklearn-color-unfitted-level-3) 1pt solid; color: var(--sklearn-

```



```

color-unfitted-level-3); } .sk-estimator-doc-link.fitted, a:link.sk-estimator-doc-link.fitted,
a:visited.sk-estimator-doc-link.fitted { /* fitted */ background-color: var(--sklearn-color-
fitted-level-0); border: var(--sklearn-color-fitted-level-3) 1pt solid; color: var(--sklearn-
color-fitted-level-3); } /* On hover */ div.sk-estimator:hover .sk-estimator-doc-link:hover,
.sk-estimator-doc-link:hover, div.sk-label-container:hover .sk-estimator-doc-link:hover,
.sk-estimator-doc-link:hover { /* unfitted */ background-color: var(--sklearn-color-
unfitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-
color-unfitted-level-0); text-decoration: none; } div.sk-estimator.fitted:hover .sk-estimator-
doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover, div.sk-label-container:hover .sk-
estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover { /* fitted */
background-color: var(--sklearn-color-fitted-level-3); border: var(--sklearn-color-fitted-
level-0) 1pt solid; color: var(--sklearn-color-fitted-level-0); text-decoration: none; } /* Span,
style for the box shown on hovering the info icon */ .sk-estimator-doc-link span { display:
none; z-index: 9999; position: relative; font-weight: normal; right: .2ex; padding: .5ex;
margin: .5ex; width: min-content; min-width: 20ex; max-width: 50ex; color: var(--sklearn-
color-text); box-shadow: 2pt 2pt 4pt #999; /* unfitted */ background: var(--sklearn-color-
unfitted-level-0); border: .5pt solid var(--sklearn-color-unfitted-level-3); } .sk-estimator-
doc-link.fitted span { /* fitted */ background: var(--sklearn-color-fitted-level-0); border:
var(--sklearn-color-fitted-level-3); } .sk-estimator-doc-link:hover span { display: block; } /*
"-specific style due to the `` HTML tag */ .sk-global a.estimator_doc_link { float: right;
font-size: 1rem; line-height: 1em; font-family: monospace; background-color: var(--
sklearn-color-unfitted-level-0); border-radius: 1rem; height: 1rem; width: 1rem; text-
decoration: none; /* unfitted */ color: var(--sklearn-color-unfitted-level-1); border: var(--
sklearn-color-unfitted-level-1) 1pt solid; } .sk-global a.estimator_doc_link.fitted { /* fitted
*/ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-
level-1) 1pt solid; color: var(--sklearn-color-fitted-level-1); } /* On hover */ .sk-global
a.estimator_doc_link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-
level-3); color: var(--sklearn-color-background); text-decoration: none; } .sk-global
a.estimator_doc_link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-
level-3); } .sk-top-container.sk-global { /* pydata-sphinx-theme hides overflow, so
scrolling is disabled. We need to set it to !important and add tabindex="0" in the HTML to
allow keyboard-only users to navigate the display. */ overflow-x: scroll !important; max-
width: 100%; } .estimator-table { font-family: monospace; } .estimator-table summary {
padding: .5rem; cursor: pointer; } .estimator-table summary::marker { font-size: 0.7rem; }
.estimator-table details[open] { padding-left: 0.1rem; padding-right: 0.1rem; padding-
bottom: 0.3rem; } .estimator-table .parameters-table { margin-left: auto !important;
margin-right: auto !important; margin-top: 0; } .estimator-table .parameters-table tr:nth-
child(odd) { background-color: #fff; } .estimator-table .parameters-table tr:nth-child(even)
{ background-color: #f6f6f6; } .estimator-table .parameters-table tr:hover td { background-
color: #e0e0e0; } .estimator-table table :is(td, th) { border: 1px solid rgba(106, 105, 104,
0.232); } /* `table td` is set in notebook with right text-align. We need to overwrite it. */
.estimator-table table td.param { text-align: left; position: relative; padding: 0; } .user-set td
{ color:rgb(255, 94, 0); text-align: left !important; } .user-set td.value { color:rgb(255, 94, 0);
background-color: transparent; } .default td, .estimator-table th { color: black; text-align:
left !important; } .user-set td i, .default td i { color: black; } td.fitted-att-type { white-space:

```

```

preserve nowrap; } /* Styles for parameter documentation links We need styling for visited
so jupyter doesn't overwrite it */ a.param-doc-link, a.param-doc-link:link, a.param-doc-
link:visited { text-decoration: underline dashed; text-underline-offset: .3em; color: inherit;
display: block; padding: .5em; } @supports(anchor-name: --doc-link) { a.param-doc-link,
a.param-doc-link:link, a.param-doc-link:visited { anchor-name: --doc-link; } } /* "hack" to
make the entire area of the cell containing the link clickable */ a.param-doc-link::before {
position: absolute; content: ""; inset: 0; } .param-doc-description { display: none; position:
absolute; z-index: 9999; left: 0; padding: .5ex; margin-left: 1.5em; color: var(--sklearn-
color-text); box-shadow: .3em .3em .4em #999; width: max-content; text-align: left; max-
height: 10em; overflow-y: auto; /* unfitted */ background: var(--sklearn-color-unfitted-
level-0); border: thin solid var(--sklearn-color-unfitted-level-3); } @supports(position-area:
center right) { .param-doc-description { position-area: center right; position: fixed; margin-
left: 0; } } /* Fitted state for parameter tooltips */ .fitted .param-doc-description { /* fitted
*/ background: var(--sklearn-color-fitted-level-0); border: thin solid var(--sklearn-color-
fitted-level-3); } .param-doc-link:hover .param-doc-description { display: block; } .copy-
paste-icon { background-image:
url(data:image/svg+xml;base64,PHN2ZyB4bWxucz0iaHR0cDovL3d3dy53My5vcmcvMjAwMC9:
background-repeat: no-repeat; background-size: 14px 14px; background-position: 0;
display: inline-block; width: 14px; height: 14px; cursor: pointer; } .features { font-family:
monospace; cursor: pointer; background-color: var(--sklearn-color-unfitted-level-0);
border: 1px dotted var(--sklearn-color-border-box); border-radius: .20em; margin-bottom:
0.5em; font-size: inherit; /* Needed for jupyter */ } .features.fitted { background-color:
var(--sklearn-color-fitted-level-0); } .features.summary { cursor: pointer; display: flex;
margin-bottom: 0; text-align: center; align-items: center; justify-content: center; gap:
0.5em; padding: .25em; } .features.details[open] > summary { color: var(--sklearn-color-
text); background-color: var(--sklearn-color-unfitted-level-2); border-radius: .20em 0 0 0; }
.features.fitted.details[open] > summary { background-color: var(--sklearn-color-fitted-
level-2); border-radius: .20em 0 0 0; } .features.details > summary .arrow::before { content:
" ▶ "; color: grey; } .features.details[open] > summary .arrow::before { content: " ▼ "; }
.features.details:hover > summary { margin: 0; background-color: var(--sklearn-color-
unfitted-level-2); } .features.fitted.details:hover > summary { margin: 0; background-color:
var(--sklearn-color-fitted-level-2); } .features .features-container { max-width: 15em; max-
height: 10em; overflow: auto; scrollbar-width: thin; padding: .25em 0.1em; background-
color: var(--sklearn-color-unfitted-level-0); border-radius: 0 0 .5em .5em; } .features.fitted
.features-container { background-color: var(--sklearn-color-fitted-level-0); } .features
.image-container { block-size: 1em; inline-size: 1em; padding: 0; margin: 0%; display: flex;
justify-content: center; align-items: center; } .features .copy-paste-icon { background-size:
1em 1em; width: 1em; height: 1em; filter: grayscale(100%) opacity(60%); } .features
.features-container table { width: 100%; margin: 0.01em; } .features .features-container
table tr:nth-child(odd) { background-color: #fff; } .features .features-container table tr:nth-
child(even) { background-color: #f6f6f6; } .features .features-container table tr:hover {
background-color: #e0e0e0; } .features .features-container table { table-layout: inherit; }
.features .features-container table td { text-align: left; padding: 0 0.5em; border: 1px solid
rgba(106, 105, 104, 0.232); white-space: nowrap; color: var(--sklearn-color-text); }
.total_features { display: flex; justify-content: center; margin-top: 0.5em; } </style> <body>

```

LinearRegression()

**In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.**

☒
fitted sk-toggleable__label-arrow">

LinearRegression

[?Documentation for LinearRegression](#)

</label>

<details> <summary>Parameters</summary> <tbody> </tbody> <tbody> </tbody>

fit_intercept fit_intercept: bool, default=True
Whether to calculate the intercept for this model. If set to False, no intercept will be used in calculations (i.e. data is expected to be centered). True

copy_X copy_X: bool, default=True
If True, X will be copied; else, it may be overwritten. True

tol tol: float, default=1e-6
The precision of the solution (coef_) is determined by tol which specifies the convergence criterion of the underlying solver. tol is set as atol and btol of :func:`scipy.sparse.linalg.lsqr` when fitting on sparse training data. tol is set as cond of :func:`scipy.linalg.lstsq` when fitting on dense training data.
.. versionadded:: 1.7
.. versionchanged:: 1.9 Now supported on dense data, interpreted as the cond parameter. 1e-06

n_jobs n_jobs: int, default=None
The number of jobs to use for the computation. This will only provide speedup in case of sufficiently large problems, that is if firstly n_targets > 1 and secondly X is sparse or if positive is set to True. None means 1 unless in a :obj:`joblib.parallel\_backend` context. -1 means using all processors. See :term:`Glossary` <n_jobs> for more details. None

positive positive: bool, default=False
When set to True, forces the coefficients to be positive. This option is only supported for dense arrays.
For a comparison between a linear regression model with positive constraints on the regression coefficients and a linear regression without such constraints, see :ref:`sphx\_glr\_auto\_examples\_linear\_model\_plot\_nnls.py`.
.. versionadded:: 0.24 False

</details>

<details> <summary>Fitted attributes</summary> <tbody> <th>Name</th>

<th>Type</th> <th>Value</th> </tbody> <tbody> </tbody>

coef_ coef_: array of shape (n_features,) or (n_targets, n_features)
Estimated coefficients for the linear regression problem. If multiple targets are passed during the fit (y 2D), this is a 2D array of shape (n_targets, n_features), while if only one target is passed, this is a 1D array of length n_features. ndarray[float64] (1,) [41933.85]

feature_names_in_ feature_names_in_: ndarray of shape (n_features_in_,)
Names of features seen during :term:`fit`. Defined only when X has feature names that are all strings.
.. versionadded:: 1.0 ndarray[object] (1,) ['median_income']

intercept_ intercept_: float or array of shape (n_targets,) Independent term in the linear model. Set to 0.0 if `fit_intercept = False`.	float64	4.446e+04
n_features_in_ n_features_in_: int Number of features seen during :term:`fit`. .. versionadded:: 0.24	int	1
rank_ rank_: int Rank of matrix `X`. Only available when `X` is dense.	int64	np.int64(1)
singular_ singular_: array of shape (min(X, y),) Singular values of `X`. Only available when `X` is dense.	ndarray[float64] (1,)	[244.69]

</details>

```

<script>/* Authors: The scikit-learn developers SPDX-License-Identifier: BSD-3-Clause */
function copyToClipboard(text, element) { // Get the parameter prefix from the closest
toggleable content const toggleableContent = element.closest('.sk-toggleable__content');
const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const
fullParamName = paramPrefix ? `paramPrefix{text}` : text; const originalStyle =
element.style; const computedStyle = window.getComputedStyle(element); const
originalWidth = computedStyle.width; const originalHTML =
element.innerHTML.replace('Copied!', ''); navigator.clipboard.writeText(fullParamName)
.then(() => { element.style.width = originalWidth; element.style.color = 'green';
element.innerHTML = "Copied!"; setTimeout(() => { element.innerHTML = originalHTML;
element.style = originalStyle; }, 2000); }) .catch(err => { console.error('Failed to copy:', err);
element.style.color = 'red'; element.innerHTML = "Failed!"; setTimeout(() => {
element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }); return false;
} document.querySelectorAll('.copy-paste-icon').forEach(function(element) { const
toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix =
toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const parent =
element.parentElement; if (!parent || !parent.nextElementSibling) { console.warn('Expected
copy-paste icon is missing from the DOM structure'); return; } const paramName =
element.parentElement.nextElementSibling.textContent.trim().split(' ')[0]; const
fullParamName = paramPrefix ? `paramPrefix{paramName}` : paramName;
element.setAttribute('title', fullParamName); }); /** * Copy the list of feature names
formatted as a Python list. * * @param {HTMLElement} element - The copy button inside a
`.features` block; its siblings * contain a `details` element and a table containing feature
named. * @returns {boolean} Always returns `false` so callers can prevent the default click
behavior. */ function copyFeatureNamesToClipboard(element) { var detailsElem =
element.closest('.features').querySelector('details'); var wasOpen = detailsElem.open;
detailsElem.open = true; var content = element.closest('.features').querySelector('tbody')
.innerHTML.trim(); if (!wasOpen) detailsElem.open = false; const rows =
content.split('\n').map(row => `row `); const formattedText = `[n{rows.join(',\n')},\n]`;
const originalHTML = element.innerHTML.replace('âœ“', ''); const originalStyle =
element.style; const copyMark = document.createElement('span'); copyMark.innerHTML =
'âœ“'; copyMark.style.color = 'blue'; copyMark.style.fontSize = '1em';
navigator.clipboard.writeText(formattedText) .then(() => { element.style.display = 'none';

```

```

element.parentElement.appendChild(copyMark); setTimeout(() => { copyMark.remove();
element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }) .catch(err =>
{ console.error('Failed to copy:', err); element.style.color = 'orange'; element.innerHTML =
"Failed!"; setTimeout(() => { element.innerHTML = originalHTML; element.style =
originalStyle; }, 1000); }); return false; } /** * Adapted from Skrub *
https://github.com/skrub-
data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228db59a31dc3/skrub/_reporting/_data/tem
* @returns "light" or "dark" */ function detectTheme(element) { const body =
document.querySelector('body'); // Check VSCode theme const themeKindAttr =
body.getAttribute('data-vscode-theme-kind'); const themeNameAttr =
body.getAttribute('data-vscode-theme-name'); if (themeKindAttr && themeNameAttr) {
const themeKind = themeKindAttr.toLowerCase(); const themeName =
themeNameAttr.toLowerCase(); if (themeKind.includes("dark") ||
themeName.includes("dark")) { return "dark"; } if (themeKind.includes("light") ||
themeName.includes("light")) { return "light"; } } // Check Jupyter theme if
(body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if
(body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based on a
parent element's color const color = window.getComputedStyle(element.parentNode,
null).getPropertyValue('color'); const match = color.match(/^rgb\s*
\s* (\d+)\s*, \s* (\d+)\s*, \s* (\d+)\s*\s*$/i); if (match) { const [r, g, b] = [
parseFloat(match[1]), parseFloat(match[2]), parseFloat(match[3]) ]; //
https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g
+ 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return
'dark'; } if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } //
Otherwise fall back to the next heuristic. } // Fallback to system preference return
window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function
forceTheme(elementId) { const estimatorElement =
document.querySelector(`#${elementId}`); if (estimatorElement === null) {
console.error(`Element with id ${elementId} not found.`); } else { const theme =
detectTheme(estimatorElement); estimatorElement.classList.add(theme); } }
forceTheme('sk-container-id-1');</script></body>

```

```

In [59]: print("Slope (Coefficient):", simple_model.coef_[0])
         print("Intercept:", simple_model.intercept_)

```

```

Slope (Coefficient): 41933.84939381274
Intercept: 44459.729169078666

```

```

In [60]: y_pred = simple_model.predict(X_test)

```

```

In [61]: from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
         import numpy as np

         print("R2 Score:", r2_score(y_test, y_pred))
         print("MAE:", mean_absolute_error(y_test, y_pred))
         print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))

```

R² Score: 0.45885918903846656
MAE: 62990.86530093761
RMSE: 84209.01241414454

```
In [62]: print("Slope:", simple_model.coef_[0])  
         print("Intercept:", simple_model.intercept_)
```

Slope: 41933.84939381274
Intercept: 44459.729169078666

```
In [63]: plt.figure(figsize=(8,6))  
  
         plt.scatter(X_test, y_test, color='blue', alpha=0.5, label='Actual Data')  
  
         plt.plot(X_test,  
                  simple_model.predict(X_test),  
                  color='red',  
                  linewidth=2,  
                  label='Regression Line')  
  
         plt.title("Simple Linear Regression")  
         plt.xlabel("Median Income")  
         plt.ylabel("Median House Value")  
         plt.grid(True)  
         plt.legend()  
  
         plt.show()
```

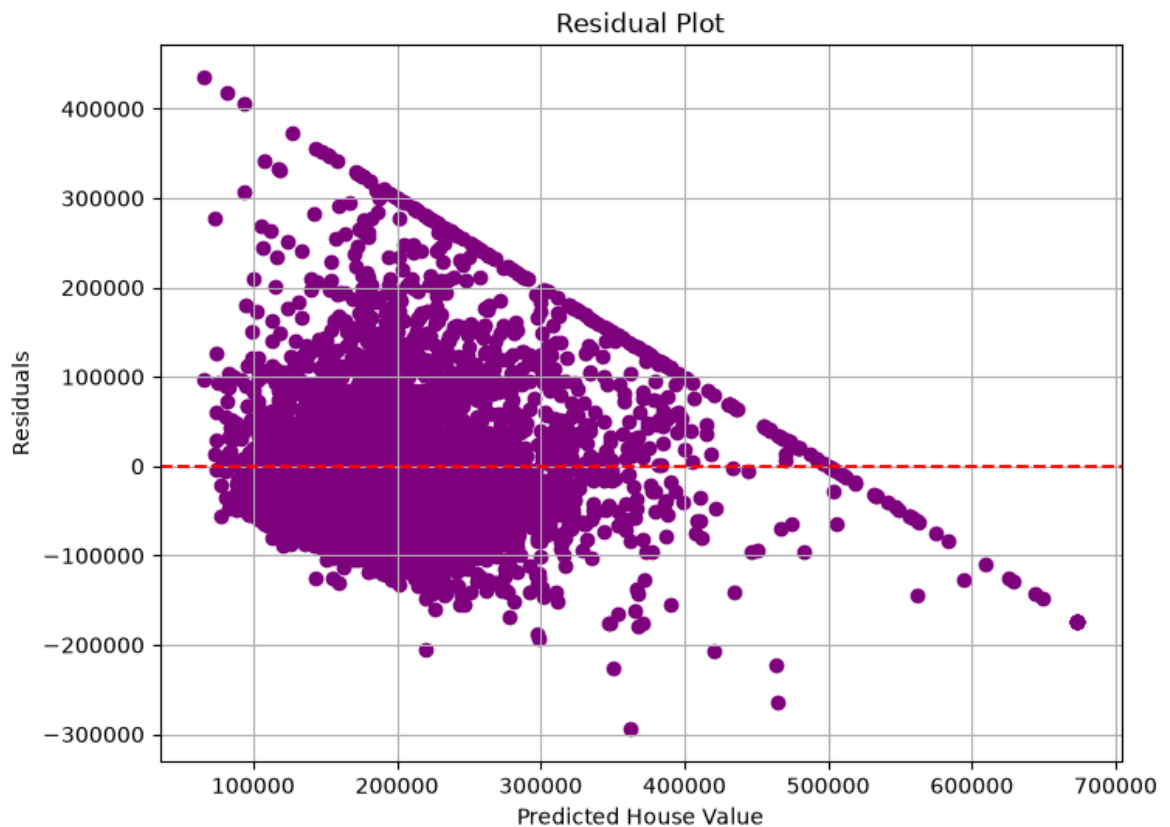


```
In [64]: residuals = y_test - y_pred  
  
         plt.figure(figsize=(8,6))  
  
         plt.scatter(y_pred, residuals, color='purple')
```

```
plt.axhline(y=0, color='red', linestyle='--')

plt.title("Residual Plot")
plt.xlabel("Predicted House Value")
plt.ylabel("Residuals")
plt.grid(True)

plt.show()
```



```
In [65]: from sklearn.ensemble import RandomForestRegressor
```

```
In [66]: rf_model = RandomForestRegressor(
            n_estimators=100,
            random_state=42
        )

rf_model.fit(X_train, y_train)
```

```

Out[66]: <style>.sk-global { /* Definition of color scheme common for light and dark mode */ --
sklearn-color-text: #000; --sklearn-color-text-muted: #666; --sklearn-color-line: gray; /*
Definition of color scheme for unfitted estimators */ --sklearn-color-unfitted-level-0:
#fff5e6; --sklearn-color-unfitted-level-1: #f6e4d2; --sklearn-color-unfitted-level-2:
#ffe0b3; --sklearn-color-unfitted-level-3: chocolate; /* Definition of color scheme for
fitted estimators */ --sklearn-color-fitted-level-0: #f0f8ff; --sklearn-color-fitted-level-1:
#d4ebff; --sklearn-color-fitted-level-2: #b3dbfd; --sklearn-color-fitted-level-3:
cornflowerblue; } .sk-global.light { /* Specific color for light theme */ --sklearn-color-text-
on-default-background: black; --sklearn-color-background: white; --sklearn-color-border-
box: black; --sklearn-color-icon: #696969; } .sk-global.dark { --sklearn-color-text-on-
default-background: white; --sklearn-color-background: #111; --sklearn-color-border-
box: white; --sklearn-color-icon: #878787; } .sk-global { color: var(--sklearn-color-text); }
.sk-global pre { padding: 0; } .sk-global input.sk-hidden--visually { border: 0; clip-path:
inset(100%); height: 1px; margin: -1px; overflow: hidden; padding: 0; position: absolute;
width: 1px; } .sk-global div.sk-dashed-wrapped { border: 1px dashed var(--sklearn-color-
line); margin: 0 0.4em 0.5em 0.4em; box-sizing: border-box; padding-bottom: 0.4em;
background-color: var(--sklearn-color-background); } .sk-global div.sk-container { /*
jupyter's `normalize.less` sets `[hidden] { display: none; }` but bootstrap.min.css set
`[hidden] { display: none !important; }` so we also need the `!important` here to be able to
override the default hidden behavior on the sphinx rendered scikit-learn.org. See:
https://github.com/scikit-learn/scikit-learn/issues/21755 */ display: inline-block
!important; position: relative; } .sk-global div.sk-text-repr-fallback { display: none; } div.sk-
parallel-item, div.sk-serial, div.sk-item { /* draw centered vertical line to link estimators */
background-image: linear-gradient(var(--sklearn-color-text-on-default-background), var(-
sklearn-color-text-on-default-background)); background-size: 2px 100%; background-
repeat: no-repeat; background-position: center center; } /* Parallel-specific style estimator
block */ .sk-global div.sk-parallel-item::after { content: ""; width: 100%; border-bottom:
2px solid var(--sklearn-color-text-on-default-background); flex-grow: 1; } .sk-global div.sk-
parallel { display: flex; align-items: stretch; justify-content: center; background-color: var(--
sklearn-color-background); position: relative; } .sk-global div.sk-parallel-item { display: flex;
flex-direction: column; } .sk-global div.sk-parallel-item:first-child::after { align-self: flex-
end; width: 50%; } .sk-global div.sk-parallel-item:last-child::after { align-self: flex-start;
width: 50%; } .sk-global div.sk-parallel-item:only-child::after { width: 0; } /* Serial-specific
style estimator block */ .sk-global div.sk-serial { display: flex; flex-direction: column; align-
items: center; background-color: var(--sklearn-color-background); padding-right: 1em;
padding-left: 1em; } /* Toggleable style: style used for
estimator/Pipeline/ColumnTransformer box that is clickable and can be
expanded/collapsed. - Pipeline and ColumnTransformer use this feature and define the
default style - Estimators will overwrite some part of the style using the `sk-estimator`
class */ /* Pipeline and ColumnTransformer style (default) */ .sk-global div.sk-toggleable {
/* Default theme specific background. It is overwritten whether we have a specific
estimator or a Pipeline/ColumnTransformer */ background-color: var(--sklearn-color-
background); } /* Toggleable label */ .sk-global label.sk-toggleable__label { cursor: pointer;
display: flex; width: 100%; margin-bottom: 0; padding: 0.5em; box-sizing: border-box;
text-align: center; align-items: center; justify-content: center; gap: 0.5em; } .sk-global

```



```

label.sk-toggleable__label.caption { font-size: 0.6rem; font-weight: lighter; color: var(--sklearn-color-text-muted); } .sk-global label.sk-toggleable__label-arrow:before { /* Arrow on the left of the label */ content: " ▶ "; float: left; margin-right: 0.25em; color: var(--sklearn-color-icon); } .sk-global label.sk-toggleable__label-arrow:hover:before { color: var(--sklearn-color-text); } /* Toggleable content - dropdown */ .sk-global div.sk-toggleable__content { display: none; text-align: left; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global div.sk-toggleable__content pre { margin: 0.2em; border-radius: 0.25em; color: var(--sklearn-color-text); /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted pre { /* unfitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global input.sk-toggleable__control:checked~div.sk-toggleable__content { /* Expand drop-down */ display: block; width: 100%; overflow: visible; } .sk-global input.sk-toggleable__control:checked~label.sk-toggleable__label-arrow:before { content: " ▼ "; } /* Pipeline/ColumnTransformer-specific style */ .sk-global div.sk-label input.sk-toggleable__control:checked~label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-label.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { background-color: var(--sklearn-color-fitted-level-2); } /* Estimator-specific style */ /* Colorize estimator box */ .sk-global div.sk-estimator input.sk-toggleable__control:checked~label.sk-toggleable__label { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } .sk-global div.sk-label label.sk-toggleable__label, .sk-global div.sk-label label { /* The background is the default theme color */ color: var(--sklearn-color-text-on-default-background); } /* On hover, darken the color of the background */ .sk-global div.sk-label:hover label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } /* Label box, darken color on hover, fitted */ .sk-global div.sk-label.fitted:hover label.sk-toggleable__label.fitted { color: var(--sklearn-color-text); background-color: var(--sklearn-color-fitted-level-2); } /* Estimator label */ .sk-global div.sk-label label { font-family: monospace; font-weight: bold; line-height: 1.2em; } .sk-global div.sk-label-container { text-align: center; } /* Estimator-specific */ .sk-global div.sk-estimator { font-family: monospace; border: 1px dotted var(--sklearn-color-border-box); border-radius: 0.25em; box-sizing: border-box; margin-bottom: 0.5em; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-estimator.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } /* on hover */ .sk-global div.sk-estimator:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } /* Specification for estimator info (e.g. "i" and "?") */ /* Common style for "i" and "?" */ .sk-estimator-doc-link, a.link.sk-estimator-doc-link, a.visited.sk-estimator-doc-link { float: right; font-size: smaller; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1em; height: 1em; width: 1em; text-decoration: none !important; margin-left: 0.5em; text-align: center; /* unfitted */ border: var(--sklearn-color-unfitted-level-3) 1pt solid; color: var(--sklearn-

```

```

color-unfitted-level-3); } .sk-estimator-doc-link.fitted, a:link.sk-estimator-doc-link.fitted,
a:visited.sk-estimator-doc-link.fitted { /* fitted */ background-color: var(--sklearn-color-
fitted-level-0); border: var(--sklearn-color-fitted-level-3) 1pt solid; color: var(--sklearn-
color-fitted-level-3); } /* On hover */ div.sk-estimator:hover .sk-estimator-doc-link:hover,
.sk-estimator-doc-link:hover, div.sk-label-container:hover .sk-estimator-doc-link:hover,
.sk-estimator-doc-link:hover { /* unfitted */ background-color: var(--sklearn-color-
unfitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-
color-unfitted-level-0); text-decoration: none; } div.sk-estimator.fitted:hover .sk-estimator-
doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover, div.sk-label-container:hover .sk-
estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover { /* fitted */
background-color: var(--sklearn-color-fitted-level-3); border: var(--sklearn-color-fitted-
level-0) 1pt solid; color: var(--sklearn-color-fitted-level-0); text-decoration: none; } /* Span,
style for the box shown on hovering the info icon */ .sk-estimator-doc-link span { display:
none; z-index: 9999; position: relative; font-weight: normal; right: .2ex; padding: .5ex;
margin: .5ex; width: min-content; min-width: 20ex; max-width: 50ex; color: var(--sklearn-
color-text); box-shadow: 2pt 2pt 4pt #999; /* unfitted */ background: var(--sklearn-color-
unfitted-level-0); border: .5pt solid var(--sklearn-color-unfitted-level-3); } .sk-estimator-
doc-link.fitted span { /* fitted */ background: var(--sklearn-color-fitted-level-0); border:
var(--sklearn-color-fitted-level-3); } .sk-estimator-doc-link:hover span { display: block; } /*
"-specific style due to the `` HTML tag */ .sk-global a.estimator_doc_link { float: right;
font-size: 1rem; line-height: 1em; font-family: monospace; background-color: var(--
sklearn-color-unfitted-level-0); border-radius: 1rem; height: 1rem; width: 1rem; text-
decoration: none; /* unfitted */ color: var(--sklearn-color-unfitted-level-1); border: var(--
sklearn-color-unfitted-level-1) 1pt solid; } .sk-global a.estimator_doc_link.fitted { /* fitted
*/ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-
level-1) 1pt solid; color: var(--sklearn-color-fitted-level-1); } /* On hover */ .sk-global
a.estimator_doc_link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-
level-3); color: var(--sklearn-color-background); text-decoration: none; } .sk-global
a.estimator_doc_link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-
level-3); } .sk-top-container.sk-global { /* pydata-sphinx-theme hides overflow, so
scrolling is disabled. We need to set it to !important and add tabindex="0" in the HTML to
allow keyboard-only users to navigate the display. */ overflow-x: scroll !important; max-
width: 100%; } .estimator-table { font-family: monospace; } .estimator-table summary {
padding: .5rem; cursor: pointer; } .estimator-table summary::marker { font-size: 0.7rem; }
.estimator-table details[open] { padding-left: 0.1rem; padding-right: 0.1rem; padding-
bottom: 0.3rem; } .estimator-table .parameters-table { margin-left: auto !important;
margin-right: auto !important; margin-top: 0; } .estimator-table .parameters-table tr:nth-
child(odd) { background-color: #fff; } .estimator-table .parameters-table tr:nth-child(even)
{ background-color: #f6f6f6; } .estimator-table .parameters-table tr:hover td { background-
color: #e0e0e0; } .estimator-table table :is(td, th) { border: 1px solid rgba(106, 105, 104,
0.232); } /* `table td` is set in notebook with right text-align. We need to overwrite it. */
.estimator-table table td.param { text-align: left; position: relative; padding: 0; } .user-set td
{ color:rgb(255, 94, 0); text-align: left !important; } .user-set td.value { color:rgb(255, 94, 0);
background-color: transparent; } .default td, .estimator-table th { color: black; text-align:
left !important; } .user-set td i, .default td i { color: black; } td.fitted-att-type { white-space:

```

```

preserve nowrap; } /* Styles for parameter documentation links We need styling for visited
so jupyter doesn't overwrite it */ a.param-doc-link, a.param-doc-link:link, a.param-doc-
link:visited { text-decoration: underline dashed; text-underline-offset: .3em; color: inherit;
display: block; padding: .5em; } @supports(anchor-name: --doc-link) { a.param-doc-link,
a.param-doc-link:link, a.param-doc-link:visited { anchor-name: --doc-link; } } /* "hack" to
make the entire area of the cell containing the link clickable */ a.param-doc-link::before {
position: absolute; content: ""; inset: 0; } .param-doc-description { display: none; position:
absolute; z-index: 9999; left: 0; padding: .5ex; margin-left: 1.5em; color: var(--sklearn-
color-text); box-shadow: .3em .3em .4em #999; width: max-content; text-align: left; max-
height: 10em; overflow-y: auto; /* unfitted */ background: var(--sklearn-color-unfitted-
level-0); border: thin solid var(--sklearn-color-unfitted-level-3); } @supports(position-area:
center right) { .param-doc-description { position-area: center right; position: fixed; margin-
left: 0; } } /* Fitted state for parameter tooltips */ .fitted .param-doc-description { /* fitted
*/ background: var(--sklearn-color-fitted-level-0); border: thin solid var(--sklearn-color-
fitted-level-3); } .param-doc-link:hover .param-doc-description { display: block; } .copy-
paste-icon { background-image:
url(data:image/svg+xml;base64,PHN2ZyB4bWxucz0iaHR0cDovL3d3dy53My5vcmcvMjAwMC9:
background-repeat: no-repeat; background-size: 14px 14px; background-position: 0;
display: inline-block; width: 14px; height: 14px; cursor: pointer; } .features { font-family:
monospace; cursor: pointer; background-color: var(--sklearn-color-unfitted-level-0);
border: 1px dotted var(--sklearn-color-border-box); border-radius: .20em; margin-bottom:
0.5em; font-size: inherit; /* Needed for jupyter */ } .features.fitted { background-color:
var(--sklearn-color-fitted-level-0); } .features.summary { cursor: pointer; display: flex;
margin-bottom: 0; text-align: center; align-items: center; justify-content: center; gap:
0.5em; padding: .25em; } .features.details[open] > summary { color: var(--sklearn-color-
text); background-color: var(--sklearn-color-unfitted-level-2); border-radius: .20em 0 0 0; }
.features.fitted.details[open] > summary { background-color: var(--sklearn-color-fitted-
level-2); border-radius: .20em 0 0 0; } .features.details > summary .arrow::before { content:
" ▶ "; color: grey; } .features.details[open] > summary .arrow::before { content: " ▼ "; }
.features.details:hover > summary { margin: 0; background-color: var(--sklearn-color-
unfitted-level-2); } .features.fitted.details:hover > summary { margin: 0; background-color:
var(--sklearn-color-fitted-level-2); } .features .features-container { max-width: 15em; max-
height: 10em; overflow: auto; scrollbar-width: thin; padding: .25em 0.1em; background-
color: var(--sklearn-color-unfitted-level-0); border-radius: 0 0 .5em .5em; } .features.fitted
.features-container { background-color: var(--sklearn-color-fitted-level-0); } .features
.image-container { block-size: 1em; inline-size: 1em; padding: 0; margin: 0%; display: flex;
justify-content: center; align-items: center; } .features .copy-paste-icon { background-size:
1em 1em; width: 1em; height: 1em; filter: grayscale(100%) opacity(60%); } .features
.features-container table { width: 100%; margin: 0.01em; } .features .features-container
table tr:nth-child(odd) { background-color: #fff; } .features .features-container table tr:nth-
child(even) { background-color: #f6f6f6; } .features .features-container table tr:hover {
background-color: #e0e0e0; } .features .features-container table { table-layout: inherit; }
.features .features-container table td { text-align: left; padding: 0 0.5em; border: 1px solid
rgba(106, 105, 104, 0.232); white-space: nowrap; color: var(--sklearn-color-text); }
.total_features { display: flex; justify-content: center; margin-top: 0.5em; } </style> <body>

```

RandomForestRegressor(random_state=42)

**In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.**

☒ >

RandomForestRegressor

[?Documentation for RandomForestRegressor](#)

</label>

<details> <summary>Parameters</summary> <tbody> </tbody> <tbody></tbody>

random_state random_state: int, RandomState instance or None, default=None

Controls both the randomness of the bootstrapping of the samples used
when building trees (if ``bootstrap=True``) and the sampling of the
features to consider when looking for the best split at each node
(if ``max\_features < n\_features``).
See :term:`Glossary` <random_state>` for details.

n_estimators n_estimators: int, default=100

The number of trees in the forest.

.. versionchanged:: 0.22
The default value of ``n\_estimators`` changed from 10 to 100
in 0.22.

criterion criterion: {"squared_error", "absolute_error", "poisson"}, default="squared_error"

The function to measure the quality of a split. Supported criteria
are "squared_error" for the mean squared error, which is equal to
variance reduction as feature selection criterion and minimizes the L2
loss using the mean of each terminal node, "absolute_error" for the mean
absolute error, which minimizes the L1 loss using the median of each terminal
node, and "poisson" which uses reduction in Poisson deviance to find splits,
also using the mean of each terminal node.

.. versionadded:: 0.18
Mean Absolute Error (MAE) criterion.

.. versionadded:: 1.0
Poisson criterion.

.. versionchanged:: 1.9
Criterion ``"friedman\_mse"`` was deprecated.

max_depth max_depth: int, default=None

The maximum depth of the tree. If None, then nodes are expanded until
all leaves are pure or until all leaves contain less than
min_samples_split samples.

min_samples_split min_samples_split: int or float, default=2

The minimum number of samples required to split an internal node:

- If int, then consider ``min\_samples\_split`` as the minimum number.
- If float, then ``min\_samples\_split`` is a fraction and
``ceil(min\_samples\_split \* n\_samples)`` are the minimum
number of samples for each split.

.. versionchanged:: 0.18
Added float values for fractions.

min_samples_leaf min_samples_leaf: int or float, default=1

The minimum number of samples required to be at a leaf node.
A split point at any depth will only be considered if it leaves at
least ``min\_samples\_leaf`` training samples in each of the left and
right branches. This may have the effect of smoothing the model,
especially in regression.

- If int, then consider ``min\_samples\_leaf`` as the minimum number.
- If float, then ``min\_samples\_leaf`` is a fraction and
``ceil(min\_samples\_leaf \* n\_samples)`` are the minimum
number of samples for each node.

.. versionchanged:: 0.18
Added float values for fractions.

min_weight_fraction_leaf min_weight_fraction_leaf: float, default=0.0

The minimum weighted fraction of the sum total of weights (of all
the input

samples) required to be at a leaf node. Samples have equal weight when `sample_weight` is not provided.

`max_features` `max_features`: {"sqrt", "log2", None}, int or float, default=1.0
The number of features to consider when looking for the best split:
- If int, then consider `max_features` features at each split.
- If float, then `max_features` is a fraction and `max(1, int(max_features * n_features_in_))` features are considered at each split.
- If "sqrt", then `max_features=sqrt(n_features)`.
- If "log2", then `max_features=log2(n_features)`.
- If None or 1.0, then `max_features=n_features`.
.. note:: The default of 1.0 is equivalent to bagged trees and more randomness can be achieved by setting smaller values, e.g. 0.3.
.. versionchanged:: 1.1 The default of `max_features` changed from "auto" to 1.0.
Note: the search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than `max_features` features.

`max_leaf_nodes` `max_leaf_nodes`: int, default=None
Grow trees with `max_leaf_nodes` in best-first fashion. Best nodes are defined as relative reduction in impurity.
If None then unlimited number of leaf nodes.

N

`min_impurity_decrease` `min_impurity_decrease`: float, default=0.0
A node will be split if this split induces a decrease of the impurity greater than or equal to this value.
The weighted impurity decrease equation is the following:
$$\frac{N_t}{N} * (\text{impurity} - \frac{N_{t_R}}{N_t} * \text{right_impurity} - \frac{N_{t_L}}{N_t} * \text{left_impurity})$$

where `N` is the total number of samples, `N_t` is the number of samples at the current node, `N_t_L` is the number of samples in the left child, and `N_t_R` is the number of samples in the right child.
`N`, `N_t`, `N_t_R` and `N_t_L` all refer to the weighted sum, if `sample_weight` is passed.
.. versionadded:: 0.19

`bootstrap` `bootstrap`: bool, default=True
Whether bootstrap samples are used when building trees. If False, the whole dataset is used to build each tree.

`oob_score` `oob_score`: bool or callable, default=False
Whether to use out-of-bag samples to estimate the generalization score.
By default, `func: ~sklearn.metrics.r2_score` is used.
Provide a callable with signature `metric(y_true, y_pred)` to use a custom metric. Only available if `bootstrap=True`.
For an illustration of out-of-bag (OOB) error estimation, see the example
:ref:`sphx\_glr\_auto\_examples\_ensemble\_plot\_ensemble\_oob.py`.

F

`n_jobs` `n_jobs`: int, default=None
The number of jobs to run in parallel.
:meth:`fit`, :meth:`predict`, :meth:`decision\_path` and :meth:`apply` are all parallelized over the trees. `None` means 1 unless in a `joblib.parallel_backend` context. -1 means using all processors. See :term:`Glossary` <n_jobs> for more details.`

N

`verbose` `verbose`: int, default=0
Controls the verbosity when fitting and predicting.

`warm_start` `warm_start`: bool, default=False
When set to `True`, reuse the solution of the previous call to `fit` and add more estimators to the ensemble, otherwise, just fit a whole new forest. See :term:`Glossary` `<warm_start>` and :ref:`tree\_ensemble\_warm\_start` for details.

F

`ccp_alpha` `ccp_alpha`: non-negative float, default=0.0
Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than `ccp_alpha` will be chosen. By default, no pruning is performed. See :ref:`minimal\_cost\_complexity\_pruning` for details.

See [:ref:`sphx\\_glr\\_auto\\_examples\\_tree\\_plot\\_cost\\_complexity\\_pruning.py`](https://github.com/scikit-learn/scikit-learn/blob/0.22.x/sphinx_glr_auto_examples/tree_plot_cost_complexity_pruning.py) for an example of such pruning.
.. versionadded:: 0.22

`max_samples` `max_samples`: int or float, default=None
If `bootstrap` is True, the number of samples to draw from `X` to train each base estimator.
If None (default), then draw `X.shape[0]` samples irrespective of `sample_weight`.
If int, then draw `max_samples` samples.
If float, then draw `max_samples * X.shape[0]` unweighted samples or `max_samples * sample_weight.sum()` weighted samples.
.. versionadded:: 0.22
.. versionchanged:: 1.9
Float `max_samples` is relative to `sample_weight.sum()` instead of `X.shape[0]` for weighted samples.

`monotonic_cst` `monotonic_cst`: array-like of int of shape (n_features), default=None
Indicates the monotonicity constraint to enforce on each feature.
- 1: monotonically increasing
- 0: no constraint
- -1: monotonically decreasing
If `monotonic_cst` is None, no constraints are applied.
Monotonicity constraints are not supported for:
- multioutput regressions (i.e. when `n_outputs_ > 1`).
Read more in the [:ref:`User Guide <monotonic\\_cst\\_gbdt>`](#).
.. versionadded:: 1.4

</details>

<details> <summary>Fitted attributes</summary> <tbody> <th>Name</th>
<th>Type</th> <th>Value</th> </tbody> <tbody> </tbody>

`estimator_` `estimator_`:
:class:`~sklearn.tree.DecisionTreeRegressor`
The child estimator template used to create the collection of fitted sub-estimators.
.. versionadded:: 1.2
`base_estimator_` was renamed to `estimator_`.

DecisionTreeRegressor

Decis

`estimators_` `estimators_`: list of DecisionTreeRegressor
The collection of fitted sub-estimators.

list

[DecisionTreeF
DecisionTreeF
DecisionTreeF
DecisionTreeF

`estimators_samples_` `estimators_samples_`: list of arrays
The subset of drawn samples (i.e., the in-bag samples) for each base estimator. Each subset is defined by an array of the indices selected.
.. versionadded:: 1.4

list

[array([31
array([47
array([54
array([2848,

`feature_importances_` `feature_importances_`: ndarray of shape (n_features,)
The impurity-based feature importances.
The higher, the more important the feature.
The importance of a feature is computed as the (normalized) total reduction of the criterion brought by that feature. It is also known as the Gini importance.
Warning: impurity-based feature importances can be misleading for high cardinality features (many unique values).
See [:func:`sklearn.inspection.permutation\\_importance`](#) as an alternative.

ndarray[float64](1,)

`feature_names_in_` `feature_names_in_`: ndarray of shape (n_features_in_,)
Names of features seen during `:term:`fit``. Defined only when `X` has feature names that are all strings.
.. versionadded:: 1.0

ndarray[object](1,)

`n_features_in_` `n_features_in_`: int
Number of features seen during `:term:`fit``.
.. versionadded::

int

`n_outputs_`: int

The number of
outputs when ``fit`` is performed.

int

</details>

```
<script>/* Authors: The scikit-learn developers SPDX-License-Identifier: BSD-3-Clause */
function copyToClipboard(text, element) { // Get the parameter prefix from the closest
toggleable content const toggleableContent = element.closest('.sk-toggleable__content');
const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const
fullParamName = paramPrefix ? `paramPrefix{text}` : text; const originalStyle =
element.style; const computedStyle = window.getComputedStyle(element); const
originalWidth = computedStyle.width; const originalHTML =
element.innerHTML.replace('Copied!', ''); navigator.clipboard.writeText(fullParamName)
.then(() => { element.style.width = originalWidth; element.style.color = 'green';
element.innerHTML = "Copied!"; setTimeout(() => { element.innerHTML = originalHTML;
element.style = originalStyle; }, 2000); }).catch(err => { console.error('Failed to copy:', err);
element.style.color = 'red'; element.innerHTML = "Failed!"; setTimeout(() => {
element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }); return false;
} document.querySelectorAll('.copy-paste-icon').forEach(function(element) { const
toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix =
toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const parent =
element.parentElement; if (!parent || !parent.nextElementSibling) { console.warn('Expected
copy-paste icon is missing from the DOM structure'); return; } const paramName =
element.parentElement.nextElementSibling.textContent.trim().split(' ')[0]; const
fullParamName = paramPrefix ? `paramPrefix{paramName}` : paramName;
element.setAttribute('title', fullParamName); }); /** * Copy the list of feature names
formatted as a Python list. * * @param {HTMLElement} element - The copy button inside a
`.features` block; its siblings * contain a `details` element and a table containing feature
named. * @returns {boolean} Always returns `false` so callers can prevent the default click
behavior. */ function copyFeatureNamesToClipboard(element) { var detailsElem =
element.closest('.features').querySelector('details'); var wasOpen = detailsElem.open;
detailsElem.open = true; var content = element.closest('.features').querySelector('tbody')
.innerHTML.trim(); if (!wasOpen) detailsElem.open = false; const rows =
content.split('\n').map(row => `row `); const formattedText = `[\n{rows.join(',\n')},\n];
const originalHTML = element.innerHTML.replace('âœ“', ''); const originalStyle =
element.style; const copyMark = document.createElement('span'); copyMark.innerHTML =
'âœ“'; copyMark.style.color = 'blue'; copyMark.style.fontSize = '1em';
navigator.clipboard.writeText(formattedText).then(() => { element.style.display = 'none';
element.parentElement.appendChild(copyMark); setTimeout(() => { copyMark.remove();
element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }).catch(err =>
{ console.error('Failed to copy:', err); element.style.color = 'orange'; element.innerHTML =
"Failed!"; setTimeout(() => { element.innerHTML = originalHTML; element.style =
originalStyle; }, 1000); }); return false; } /** * Adapted from Skrub *
https://github.com/skrub-
data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228db59a31dc3/skrub/_reporting/_data/tem
* @returns "light" or "dark" */ function detectTheme(element) { const body =
```

```

document.querySelector('body'); // Check VSCode theme const themeKindAttr =
body.getAttribute('data-vscode-theme-kind'); const themeNameAttr =
body.getAttribute('data-vscode-theme-name'); if (themeKindAttr && themeNameAttr) {
const themeKind = themeKindAttr.toLowerCase(); const themeName =
themeNameAttr.toLowerCase(); if (themeKind.includes("dark") ||
themeName.includes("dark")) { return "dark"; } if (themeKind.includes("light") ||
themeName.includes("light")) { return "light"; } } // Check Jupyter theme if
(body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if
(body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based on a
parent element's color const color = window.getComputedStyle(element.parentNode,
null).getPropertyValue('color'); const match = color.match(/^rgb\s*
\s* (\d+)\s*, \s* (\d+)\s*, \s* (\d+)\s*\s*$/i); if (match) { const [r, g, b] = [
parseFloat(match[1]), parseFloat(match[2]), parseFloat(match[3]) ]; //
https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g
+ 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return
'dark'; } if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } //
Otherwise fall back to the next heuristic. } // Fallback to system preference return
window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function
forceTheme(elementId) { const estimatorElement =
document.querySelector(`#${elementId}`); if (estimatorElement === null) {
console.error(`Element with id ${elementId} not found.`); } else { const theme =
detectTheme(estimatorElement); estimatorElement.classList.add(theme); } }
forceTheme('sk-container-id-2');
```

```
In [67]: rf_pred = rf_model.predict(X_test)
```

```
In [68]: from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np

rf_r2 = r2_score(y_test, rf_pred)
rf_mae = mean_absolute_error(y_test, rf_pred)
rf_rmse = np.sqrt(mean_squared_error(y_test, rf_pred))

print("Random Forest Results")
print("-----")
print("R² Score :", rf_r2)
print("MAE :", rf_mae)
print("RMSE :", rf_rmse)
```

```

Random Forest Results
-----
R² Score : 0.28171270917366753
MAE : 71316.702948597
RMSE : 97018.04547892926
```

```
In [69]: importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf_model.feature_importances_
})

importance = importance.sort_values(
```



```

    by="Importance",
    ascending=False
)

importance

```

```

Out[69]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>Feature</th> <th>Importance</th> </thead> <tbody> <tr> <td>0</td>
</tbody> <tbody> </tbody>

median_income    1.0

```

```

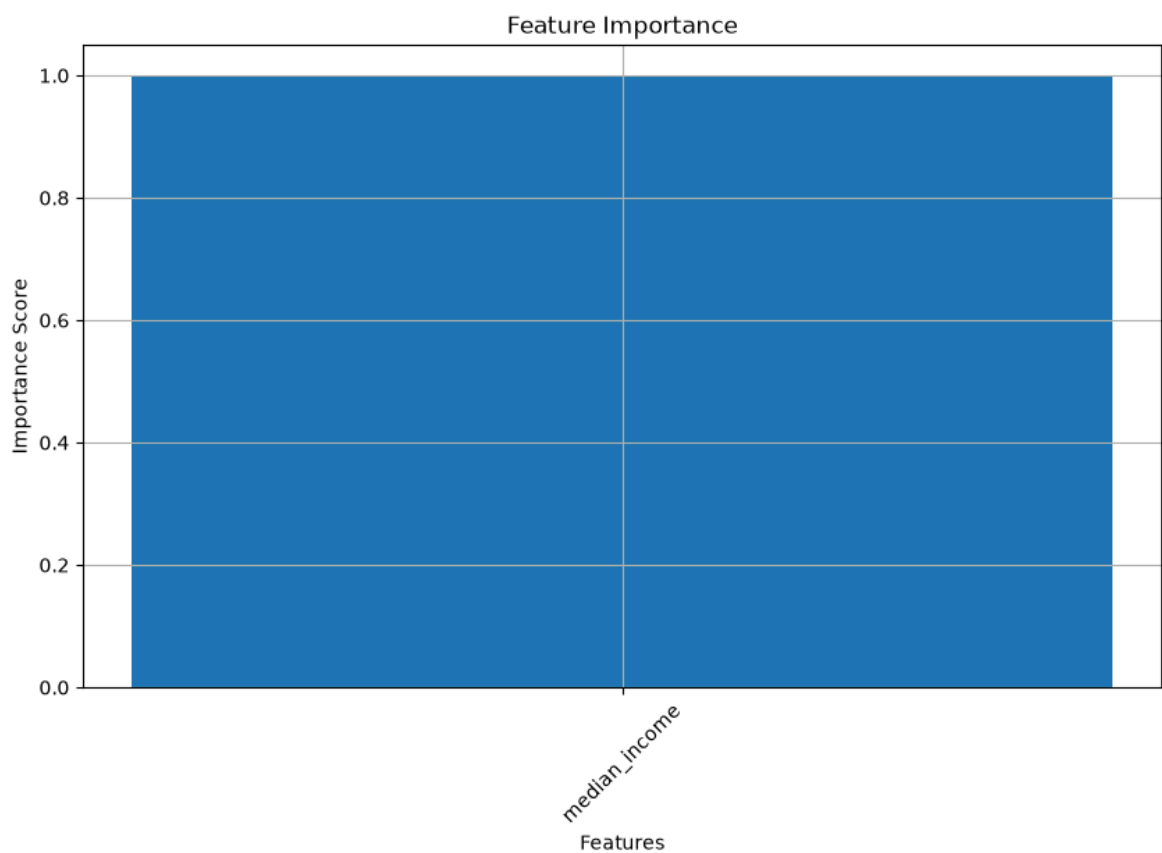
In [70]: plt.figure(figsize=(10,6))

plt.bar(
    importance["Feature"],
    importance["Importance"]
)

plt.title("Feature Importance")
plt.xlabel("Features")
plt.ylabel("Importance Score")
plt.xticks(rotation=45)
plt.grid(True)

plt.show()

```



```

In [71]: import plotly.express as px

fig = px.histogram(
    df,

```

```

    x="median_house_value",
    nbins=30,
    title="Interactive Histogram of Median House Value",
    color_discrete_sequence=["skyblue"]
)

fig.update_layout(
    xaxis_title="Median House Value",
    yaxis_title="Count"
)

fig.show()

```

```

In [72]: fig = px.scatter(
    df,
    x="median_income",
    y="median_house_value",
    color="housing_median_age",
    title="Median Income vs Median House Value",
    hover_data=["population", "households"]
)

fig.update_layout(
    xaxis_title="Median Income",
    yaxis_title="Median House Value"
)

fig.show()

```

```

In [73]: import plotly.express as px

corr = df.corr(numeric_only=True)

fig = px.imshow(
    corr,
    text_auto=True,
    color_continuous_scale="RdBu_r",
    title="Interactive Correlation Heatmap"
)

fig.show()

```

```

In [74]: import notebook
print(notebook.__version__)

```

7.6.0

```

In [75]: import plotly
print(plotly.__version__)

```

6.9.0

```

In [76]: import plotly.io as pio
pio.renderers.default = "browser"

```

```

In [77]: import plotly.express as px

fig = px.scatter(
    x=[1, 2, 3, 4],

```

```
y=[10, 15, 13, 17],
title="Plotly Test"
)

fig.show()
```

```
In [3]: import matplotlib.pyplot as plt

summary.plot(kind='bar', figsize=(12,6))

plt.title("Comparison of Mean, Median and Mode")
plt.xlabel("Features")
plt.ylabel("Values")
plt.xticks(rotation=45)
plt.grid(True)

plt.show()
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[3], line 3
      1 import matplotlib.pyplot as plt
      2
----> 3 summary.plot(kind='bar', figsize=(12,6))
      4
      5 plt.title("Comparison of Mean, Median and Mode")
      6 plt.xlabel("Features")

NameError: name 'summary' is not defined
```

```
In [4]: import pandas as pd

df = pd.read_csv("housing.csv")
```

```
In [5]: mean_values = df.mean(numeric_only=True)

median_values = df.median(numeric_only=True)

mode_values = df.mode().iloc[0]

summary = pd.DataFrame({
    "Mean": mean_values,
    "Median": median_values,
    "Mode": mode_values
})

summary
```

```
Out[5]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>Mean</th> <th>Median</th> <th>Mode</th> </thead> <tbody>
<th>households</th> <th>housing_median_age</th> <th>latitude</th>
<th>longitude</th> <th>median_house_value</th> <th>median_income</th>
<th>ocean_proximity</th> <th>population</th> <th>total_bedrooms</th>
<th>total_rooms</th> </tbody> <tbody></tbody>
```

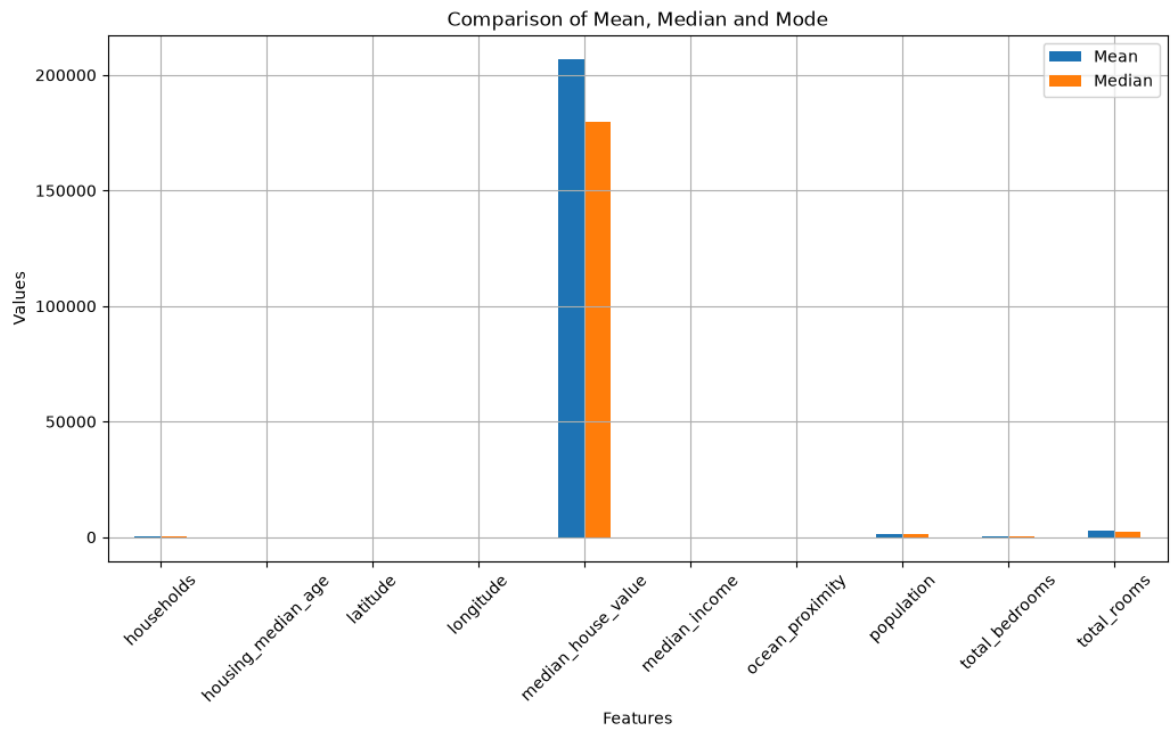
499.539680	409.0000	306.0
28.639486	29.0000	52.0
35.631861	34.2600	34.06
-119.569704	-118.4900	-118.31
206855.816909	179700.0000	500001.0
3.870671	3.5348	3.125
NaN	NaN	<1H OCEAN
1425.476744	1166.0000	891.0
537.870553	435.0000	280.0
2635.763081	2127.0000	1527.0

```
In [6]: import matplotlib.pyplot as plt

summary.plot(kind='bar', figsize=(12,6))

plt.title("Comparison of Mean, Median and Mode")
plt.xlabel("Features")
plt.ylabel("Values")
plt.xticks(rotation=45)
plt.grid(True)

plt.show()
```

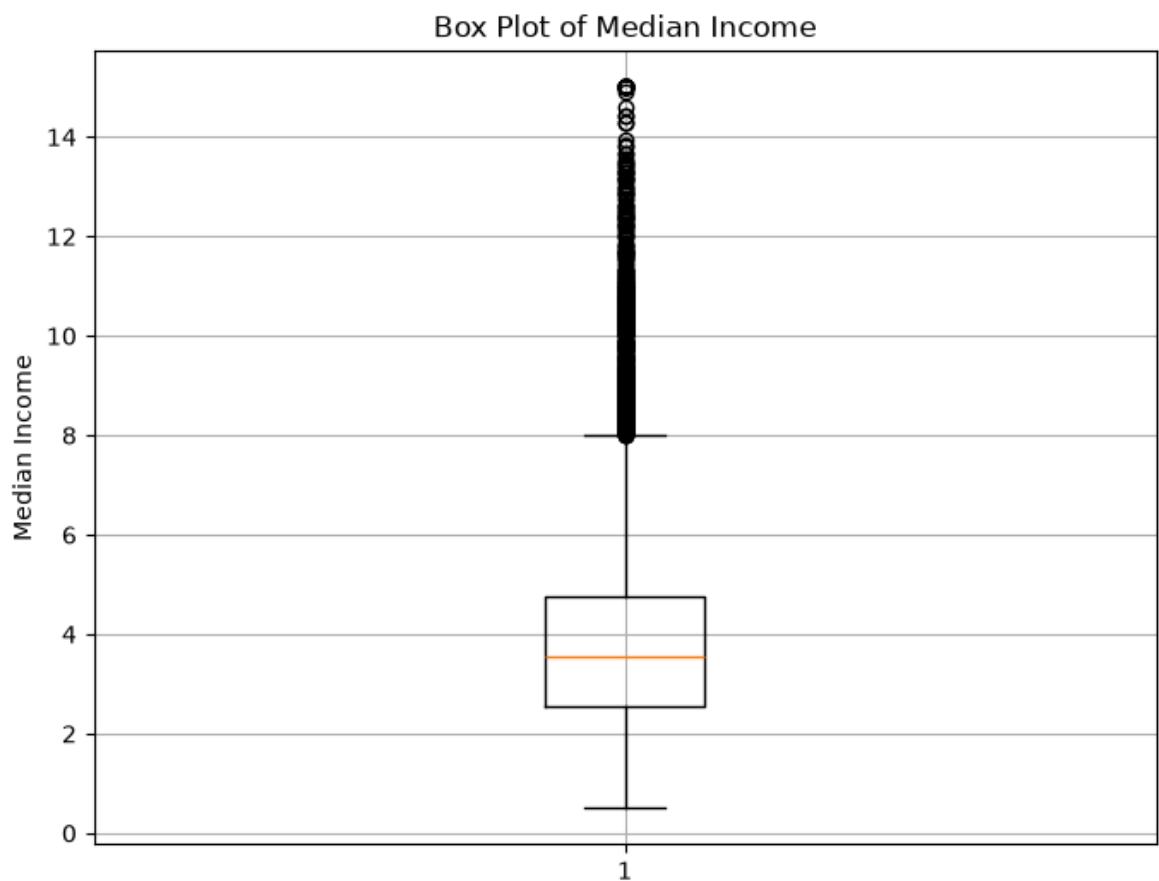


```
In [7]: plt.figure(figsize=(8,6))

plt.boxplot(df["median_income"])

plt.title("Box Plot of Median Income")
plt.ylabel("Median Income")
plt.grid(True)

plt.show()
```



```
In [8]: values = [
        df["median_income"].mean(),
        df["median_income"].median(),
        df["median_income"].mode()[0]
        ]

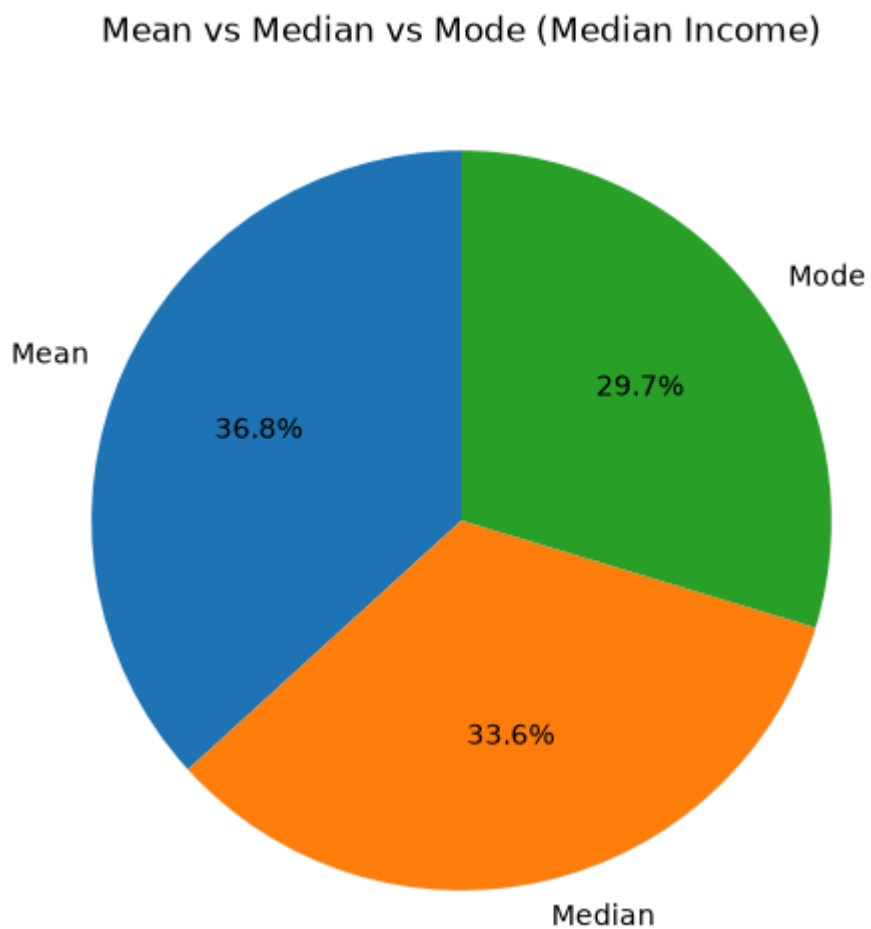
labels = ["Mean", "Median", "Mode"]

plt.figure(figsize=(6,6))

plt.pie(values,
        labels=labels,
        autopct="%1.1f%%",
        startangle=90)

plt.title("Mean vs Median vs Mode (Median Income)")

plt.show()
```



In []: